ชีทสอบปฏิบัติ **Backend** สำหรับผู้เริ่มต้น

Node.js Express MySQL / Open book แบบไม่ใช้อินเทอร์เน็ต

ภาษาไทยพร้อมคำสั่ง โค้ด ตัวอย่าง request และวิธีแก้ error | 22 ตุลาคม 2026

บท 01 เริ่มใช้ชีทจากตรงนี้

ชีทนี้ใช้ทำข้อสอบเขียน Backend ด้วย Node.js, Express และ MySQL แบบเปิดหนังสือและไม่ใช้อินเทอร์เน็ต เริ่มจากอ่านโจทย์ให้รู้ว่าต้องรับอะไร คืนอะไร แล้วแก้ทีละไฟล์ โค้ดเต็มอยู่ภาคท้ายและในโฟลเดอร์ practice-project ของชุดไฟล์

เวลาสอบให้ยึด **spec** และ **template** ที่อาจารย์แจกเป็นหลัก ชุดโค้ดประกอบเป็นเฉลยสำหรับแบบฝึก **Mobile Distributor** ในชีทนี้ ชื่อตาราง **field** รูปแบบ **JSON** และเงื่อนไขของข้อสอบจริงอาจต่างออกไป

ถ้าเริ่มไม่ถูก ให้ทำตามลำดับนี้

1. เปิดโฟลเดอร์ที่มี package.json ใน VS Code แล้วเปิด Terminal ในโฟลเดอร์นั้น
2. อ่านโจทย์และ SQL แล้วกรอกตารางในบท 03 ก่อนเริ่มคัดลอกโค้ด
3. ตั้งรหัสนักศึกษา ชื่อ secretKey และการเชื่อมต่อ MySQL ให้ตรงโจทย์
4. รัน entry file ที่แก้จริง เช่น node src/app.js แล้วเปิด student-info เพื่อเช็กเส้นทาง
5. ทำ GET ทีละรายการก่อน แล้วทำ list และ pagination จากนั้น POST, PUT และ DELETE
6. ทดสอบทั้งกรณีสำเร็จและกรณีผิด พร้อมตรวจชื่อ field ชนิดข้อมูล และสถานะ HTTP

อาการหรือคำในโจทย์	เปิดบท
ยังไม่ได้เตรียมเครื่อง หรือไม่แน่ใจว่าปิดเน็ตแล้วรันได้ไหม	02 (หน้า 2)
ไม่รู้เริ่มไฟล์ไหน ใช้ ZIP ไหน หรือรันคำสั่งอะไร	04 (หน้า 4)
ไม่เข้าใจ req, res, route, service, repository, mapper	05-06 (หน้า 6-7)
ไม่รู้ชื่อตาราง PK หรือ brand_id ต่างจาก brandId อย่างไร	07 (หน้า 9)
success / error / timestamp / status code	08 (หน้า 11)
GET by id / ไม่พบข้อมูล 404	09 (หน้า 12)
GET list / page / limit / search / filter / total	10 (หน้า 14)
Detail DTO / JOIN / brandName / วันที่	11 (หน้า 16)

อาการหรือคำในโจทย์	เปิดบท
POST / required / duplicate / foreign key	12 (หน้า 18)
PUT / full update / ข้อมูลเหมือนเดิม	13 (หน้า 20)
DELETE / 204 / RESOURCE_IN_USE	14 (หน้า 21)
โจทย์เปลี่ยนตาราง view หรือชื่อ response	15 (หน้า 22)
PK ข้อความ PK คู่ PATCH sort nested aggregate	16 (หน้า 24)
ส่ง request ไม่เป็น หรือเทียบผลไม่เป็น	17 (หน้า 26)
รันพัง 500 import error SQL error	18 (หน้า 28)
เหลือเวลาไม่มาก อยากตรวจให้ครบก่อนส่ง	19 (หน้า 30)
ต้องการโค้ดครบไฟล์สำหรับแบบฝึก	ภาคผนวก A หน้า 33

เปิด HTML ด้วยเบราว์เซอร์ในเครื่องได้โดยไม่ต้องเริ่ม server ใช้ช่องค้นหาหรือ Ctrl+F และปุ่มคัดลอกโค้ด PDF เหมาะกับเปิดดูหรือพิมพ์ หากคัดลอกจาก PDF แล้ววรรคหรืออัญประกาศเพี้ยน ให้ใช้ HTML หรือไฟล์ .js ในชุดไฟล์แทน

อ้างอิง แนวทางการใช้ชีทที่จัดทำเพิ่มจากไฟล์แนบ

บท 02 เตรียมให้ใช้งานได้ก่อนตัดอินเทอร์เน็ต

ตัวชีท PDF และ HTML เปิดออฟไลน์ได้ แต่โปรเจกต์ Express ต้องมี Node.js แพ็กเกจ npm และฐานข้อมูลพร้อมอยู่ในเครื่องก่อน ZIP ต้นฉบับที่แนบมาไม่มี node_modules จึงยังไม่ใช่โปรเจกต์ที่แตกแล้วรันได้ทันที

ทำก่อนวันสอบหรือก่อนตัดอินเทอร์เน็ต

1. เตรียม Node.js, VS Code และ MySQL ที่ใช้จริง เปิดโปรแกรมให้ได้ เก็บ SQL และคำสั่งจากอาจารย์ไว้ในเครื่อง

2. แตก ZIP และเปิดโฟลเดอร์ชั้นที่มี package.json ไม่ใช่ชั้นนอกที่มีแต่ชื่อโฟลเดอร์ซ้อนกัน

3. ตรวจ node -v และ npm -v ถ้าไม่รู้จักคำสั่ง ให้แก้การติดตั้งหรือ PATH ก่อนวันสอบ

4. ขณะยังมีอินเทอร์เน็ต รัน npm ci ในโปรเจกต์ที่มี package-lock.json ตรงกับ package.json ถ้าไม่มี lockfile ใช้ npm install ตามสภาพโปรเจกต์

5. เก็บทั้งโปรเจกต์รวม node_modules ไว้ตามกติกาที่อนุญาต แล้วตั้ง DB และทดสอบ endpoint จริง

6. ปิดอินเทอร์เน็ตและทดลองอีกครั้ง เปิด HTML เริ่ม Node เรียก localhost อ่านข้อมูล และสร้างข้อมูลในฐานฝึกให้ได้

Terminal ก่อนตัดอินเทอร์เน็ตเท่านั้น

```
node -v
npm -v
npm ci
```

อย่ารัน **npm ci** ซ้ำเพื่อซ่อมโปรเจกต์ระหว่างออฟไลน์บนสำเนาที่พร้อมใช้เพียงชุดเดียว คำสั่งนี้ลบ **node_modules** เดิมก่อนติดตั้งใหม่ และอาจติดตั้งกลับไม่ได้เมื่อไม่มีแพ็กเกจใน **cache** ครบ **npm install --offline** ก็ต้องมี **cache** ครบเช่นกัน

สิ่งที่ต้องมี	ตรวจอย่างไร
Node.js	node -v แสดงเวอร์ชัน Express 5.2.1 ใน lockfile ต้นฉบับต้องการ Node >= 18
Dependencies	มี node_modules และเริ่มแอปจริงแล้วไม่ขึ้น Cannot find package
MySQL	เปิด service และใช้ host/port/user/password ตามเครื่องหรือใบงาน
ฐานและตาราง	นำเข้า SQL ที่กำหนดแล้ว คำสั่ง SELECT อ่านแถวได้
ไฟล์โจทย์และตัวอย่าง	ดาวน์โหลดเก็บไว้จริง เปิดได้โดยไม่พึ่งลิงก์เว็บไซต์
เครื่องมือยิง request	ใช้ Postman โหมดที่เตรียมออฟไลน์ไว้ หรือ curl.exe บน Windows
ชีท	ดาวน์โหลด PDF/HTML/ZIP แล้วลองเปิดจากไฟล์ในเครื่อง

localhost และ 127.0.0.1 หมายถึงเครื่องตัวเอง จึงเรียก API ในเครื่องได้แม้ไม่มีอินเทอร์เน็ต ถ้าฐานข้อมูลอยู่เครื่องอื่น ต้องใช้เครือข่ายห้องสอบที่ได้รับอนุญาตตามใบงาน การพิมพ์ว่า Server running เพียงอย่างเดียวยังไม่ยืนยันว่าเชื่อม MySQL สำเร็จ ต้องลอง GET ที่อ่านฐานด้วย

ไม่มี SQL ของฐานสอบจริงและไม่มี tests/test-runner.mjs อยู่ในชุดแนบนี้ ชุดไฟล์ที่จัดทำเพิ่มจึงมีฐานฝึกและ unit tests ของตัวเอง หากอาจารย์แจก SQL หรือ test runner ตอนสอบ ให้ใช้ไฟล์นั้นตามคำสั่งจริง

อ้างอิง ตรวจ package.json **และ** package-lock.json **ใน** ZIP template **ทั้งสามชุด** [S2-S4] **และ** Backend_ExamData [S5]

บท 03 อ่านโจทย์แล้วแปลงเป็นรายการที่ต้องทำ

อย่าเริ่มจากเลือกโค้ดที่หน้าตาคล้ายกัน เริ่มจากทำตารางของโจทย์หนึ่งข้อให้ครบก่อน เพราะชื่อ field และกติกาที่ต่างเพียงจุดเดียวอาจทำให้ผลตรวจไม่ตรง

รายการ	ต้องจดจากโจทย์
Method และ path	GET /api/รหัสนักศึกษา/products/:id หรือ POST /products เป็นต้น
รับค่าจากไหน	params เช่น id / query เช่น page / body เช่น sku และ price
ฐานข้อมูล	database, table, PK, AUTO_INCREMENT, UNIQUE, FK, NULL, DEFAULT
ผลสำเร็จ	สถานะ 200/201/204 และ JSON ตัวอย่างครบทุกชั้น
Fields และชนิด	id เป็น number หรือ string, price เป็น number, วันที่ใช้รูปแบบใด

รายการ	ต้องจดจากโจทย์
กรณีผิด	required, ค่าติดลบ, ไม่พบข้อมูล, ซ้ำ, FK และ code/message ที่กำหนด
การแบ่งหน้า	ค่า default, เพดาน limit, กรณีค่าไม่ถูก, ชื่อ metadata
ลำดับตรวจ	ถ้าผิดหลายอย่างพร้อมกัน ต้องตอบ error ของเงื่อนไขใดก่อน

ตัวอย่างแปลงประโยคในโจทย์เป็นงาน

โจทย์บอก	จึงต้องทำ
แสดงสินค้าหนึ่งชิ้นตาม id	route GET /:id, อ่าน req.params.id, SQL WHERE id = ?
ไม่พบให้ตอบ 404	ตรวจ rows[0] ก่อนเข้า mapper แล้ว throw AppError
แสดง brandName	JOIN brands และตั้ง alias ให้ mapper อ่านได้
แสดงเฉพาะ 7 fields	สร้าง object ใหม่ใน summary mapper ห้ามส่ง SELECT * ออกทั้งแถว
sku ห้ามซ้ำ	ฐานต้องมี unique และแปลง ER_DUP_ENTRY เป็น 409 ตาม spec
stock_quantity เป็น 0 ได้	ตรวจ < 0 อย่าใช้ if (!stock_quantity)
แบ่งหน้าหลังค้นหา	SELECT และ COUNT ต้องใช้ WHERE ชุดเดียวกัน
DELETE สำเร็จ 204	res.status(204).end() ไม่ห่อ JSON

ชื่อในฐาน ชื่อใน **request** และชื่อใน **response** เป็นคนละเรื่อง ตัวอย่างแบบฝึกนี้รับ **body.brand_id** แต่คืน **data.brandId** หากโจทย์จริงรับ **brandId** ต้องเพิ่มการแปลงใน **service** หรือ **validator** ไม่ใช่เปลี่ยนเฉพาะชื่อ **output**

สัญญาสำหรับแบบฝึกในชีทนี้ใช้ 6 ข้อจาก HTML ที่แนบ ไม่ใช่คำยืนยันจำนวนข้อหรือเนื้อหาข้อสอบจริง หัวข้อดัดแปลงในบท 16 เป็นรูปแบบที่เตรียมฝึกได้ ไม่ใช่การทำนายว่าออกแน่นอน

อ้างอิง HTML **แบบฝึก** [S1] **และเอกสารคำตอบ** [S6-S7]

บท 04 เลือกไฟล์ที่แก้และรันโปรเจกต์ให้ถูกตัว

ชุดไฟล์	จุดที่ต้องรู้ก่อนใช้
newest-truely-midterm-express-esm-template.zip	มี src/exam-app.js และ helper exam-* ภายใต้โฟลเดอร์ลงท้าย -$id ยังต้องแก้ placeholder และ DTO
truely-midterm-express-esm-template.zip	โค้ด helper ใกล้เคียง newest ความต่างหลักคือคู่มือและคำอธิบาย

ชุดไฟล์	จุดที่ต้องรู้ก่อนใช้
incase-of-that-shit-template-not-work.zip	โฟลเดอร์ไม่มี -$id ใช้ exam-crud-* response และกฎ pagination ต่างจาก newest
Backend_ExamData-main.zip	มี mockExam, template ซ้ำ และสไลด์ ตัวอย่าง mockExam ยังมีจุดผิดและ resource เดิมค้าง
practice-project ในชุดไฟล์ใหม่นี้	เป็นตัวอย่างครบชุดสำหรับแบบฝึกในชีทนี้ ใช้ src/app.js และฐานฝึกแยก

คำสั่งเริ่มระบบ

สำหรับชุดฝึกใหม่ ให้แตก ZIP แล้วเปิด VS Code เลือก File > Open Folder > เลือกโฟลเดอร์ practice-project จากนั้น Terminal > New Terminal ตรวจใน Explorer ว่าเห็น package.json แล้วพิมพ์คำสั่งทีละบรรทัด พิมพ์เฉพาะคำสั่ง ไม่ต้องพิมพ์ > หรือ PS ที่แสดงอยู่หน้า Terminal

ชุดฝึกใหม่ หรือไฟล์ app.js ของอาจารย์ที่แก้แล้ว

```
node src/app.js
```

template ต้นฉบับ เมื่อเลือกใช้และแก้ exam-app.js แล้ว

```
node src/exam-app.js
```

1. เปิด Terminal จากโฟลเดอร์ที่เห็น package.json และ src ถ้าอยู่ผิดที่ใช้ cd เข้าโฟลเดอร์ให้ถูกก่อน

2. รันเพียง entry file ที่ใช้จริง อย่าเปิดสอง server บน port 3000 พร้อมกัน

3. เมื่อแก้แล้วกด Ctrl+S ถ้าไม่ได้ใช้ watch mode ให้ Ctrl+C เพื่อหยุด จากนั้นรันคำสั่งเดิมใหม่

4. ปล่อย Terminal แรกให้ server ทำงาน ใช้อีก Terminal หรือ Postman ส่ง request

package.json ของ **ZIP** ต้นฉบับไม่มี **npm start** หรือ **npm run dev** และ **npm test** เป็นคำสั่ง **placeholder** ที่ **fail** ส่วน **practice-project** ใหม่มี **npm start** และ **npm test** ที่กำหนดไว้จริง อย่าจำคำสั่งข้ามชุด

src/test-service.js ใน template ไม่ใช่ตัวเริ่ม server และไม่ใช่ test runner ของข้อสอบ บางชุด import ไฟล์ที่ไม่มีและมีคำสั่งลบข้อมูล จึงไม่ใช้เป็นขั้นตอนลองรัน

ถ้าโฟลเดอร์มีชื่อ -$id

$id ในชื่อโฟลเดอร์เป็นตัวอักษรจริง ถ้าโจทย์สั่งเปลี่ยนเป็นรหัสนักศึกษา ให้ Rename ใน VS Code แล้วใช้ Ctrl+Shift+F หา -$id เพื่อตรวจทุก import ให้ตรงกัน ห้ามแก้เฉพาะชื่อโฟลเดอร์แล้วปล่อย import เดิมไว้ ถ้าทำผ่าน Bash หรือ PowerShell ให้ใช้อัญประกาศเดี่ยว เช่น 'src/routes-$id/route.js' เพื่อไม่ให้ $id ถูกแทนค่า การ Rename ผ่าน VS Code จะตรงไปตรงมากว่า

ใช้โครงที่อาจารย์บังคับก่อน หากโจทย์ไม่ได้บังคับชื่อไฟล์ ให้รักษาโครงเดียวตลอดคำตอบ ชุดฝึกในชีทนี้ใช้ routes, services, repositories, mappers, middlewares, configs และ validators โดยไม่มี -$id

อ้างอิง audit **โครงสร้าง** ZIP [S2-S5]

บท 05 เข้าใจหน้าที่ของไฟล์และค่าที่ส่งเข้ามา

Backend คือโปรแกรมที่รอรับคำขอ เช่น ขอข้อมูลสินค้า แล้วอ่านหรือแก้ฐานข้อมูลและส่งผลกลับ API คือเส้นทางที่โปรแกรมอื่นเรียกได้ เช่น GET /products/1 ส่วน endpoint คือ method กับ URL ที่จับคู่กัน

ไฟล์หรือคำศัพท์	หน้าที่	ตัวอย่างในชุดฝึก
app	เริ่ม Express ตั้ง middleware และต่อ router เข้ากับ URL	src/app.js
route	รับ HTTP อ่าน input เรียก service และส่ง status/response	routes/product-route.js
service	ตรวจเงื่อนไข เช่น ไม่พบสินค้า หรือ brand ไม่มี	services/product-service.js
repository	เขียน SQL และติดต่อ MySQL	repositories/product-repo.js
mapper หรือ DTO	เลือกและเปลี่ยนชื่อ field ก่อนส่งให้ผู้เรียก	mappers/product-mapper.js
middleware	ทำงานระหว่างทาง เช่น อ่าน JSON แบ่งหน้า จัด error	middlewares/error-handler.js
validator	เช็ก body และเลือก field ที่อนุญาต	validators/product-validator.js
config	ค่าที่เปลี่ยนตามเครื่องหรือผู้สอบ	configs/db-config.js

คำขอผ่าน app ไป route จากนั้น service เรียก repository เพื่ออ่านฐาน เมื่อได้แถวกลับมา service ตรวจและ map ข้อมูล ก่อน route ส่ง response อย่าใส่ SQL ไว้ใน mapper และอย่าใช้ res.json ใน repository

ค่าที่ Express ให้มา

```
// URL: /api/68123456789/products/7?page=2&limit=10
req.params.id     // '7' จาก /:id
req.query.page    // '2' จาก ?page=2
req.query.limit   // '10' จาก &limit=10
req.body          // JSON ที่ส่งด้วย POST หรือ PUT
```

ตัวแปร	ความหมาย
req	request ข้อมูลที่ผู้เรียกส่งเข้ามา
res	response เครื่องมือส่งคำตอบกลับ เช่น status และ json
next()	ส่งต่อให้ middleware ถัดไป

ตัวแปร	ความหมาย
next(error)	ส่งปัญหาไป error middleware
params	ค่าที่แทน :id หรือชื่อ parameter ใน path
query	เงื่อนไขหลัง ? ใน URL ส่วนใหญ่เป็นข้อความ ต้องตรวจชนิดก่อนใช้
body	ข้อมูล JSON ต้องมี express.json() ก่อน router และ Content-Type ถูกต้อง

ตัวอย่าง 68123456789 ในชีทเป็นรหัสสมมติสำหรับอธิบาย URL เท่านั้น ต้องแทนด้วยค่าจริงที่โจทย์กำหนดทุกครั้ง

อ้างอิง สไลด์ HTTP REST Express **และ** Error Handling **ใน** [S5]

บท 06 อ่าน JavaScript และ SQL ที่ต้องใช้ให้ออก

โค้ด	อ่านว่า
import * as repo from '../repositories/product-repo.js'	นำฟังก์ชันที่ export มาไว้ใน object repo
export async function findOne(id)	ประกาศฟังก์ชันที่ไฟล์อื่นเรียกได้และทำงานแบบรอผล
await repo.findOne(id)	รอผลจริงจากฐานก่อนทำบรรทัดถัดไป
const [rows] = await db.query(sql, values)	เอาสมาชิกตัวแรกจาก [rows, fields] ของ mysql2
rows	array ของหลายแถว เช่น [{id:1}, {id:2}]
rows[0]	แถวแรก ถ้าไม่พบจะเป็น undefined
rows[0] ?? null	ถ้าไม่มีแถวแรกให้คืน null
products.map(toSummaryDTO)	แปลงสินค้าแต่ละแถวให้เป็น DTO แล้วได้ array ใหม่
throw new AppError(...)	หยุดเส้นทางปกติและส่งข้อผิดพลาดขึ้นไป
try / catch	ลองทำงาน แล้วจัดการ error ที่เกิดในงานนั้น
return	ส่งค่ากลับไปให้ฟังก์ชันที่เรียก และจบฟังก์ชันนี้

ตัวอย่างอ่านหนึ่งรายการทีละบรรทัด

repository ตัวอย่าง

```
export async function findOne(id) {
  const [rows] = await db.query(
    'SELECT * FROM brands WHERE id = ?',
    [id]
  )
  return rows[0] ?? null
}
```

บรรทัดแรกสร้างฟังก์ชันรับ id ส่วน query สั่งฐานหา brand ที่มี id ตรงกัน เครื่องหมาย ? คือช่องสำหรับค่า [id] ที่แยกจาก SQL ผล query เป็นรายการแถว จึงเลือก rows[0] ถ้าไม่มีแถวให้ service เป็นคนตัดสิน 404

SQL	หน้าที่
SELECT … FROM table WHERE id = ?	อ่านข้อมูลตามเงื่อนไข
ORDER BY id ASC	เรียงจากน้อยไปมากให้แบ่งหน้าได้คงที่
LIMIT ? OFFSET ?	หยิบจำนวนที่กำหนดและข้ามแถวก่อนหน้า
COUNT(*) AS total	นับจำนวนแถวและตั้งชื่อผลว่า total
INSERT INTO table SET ?	สร้างแถวจาก object ใช้กับ mysql2.query
UPDATE table SET ? WHERE id = ?	แก้แถวตาม id ด้วย object ของค่าที่อนุญาต
DELETE FROM table WHERE id = ?	ลบแถวตาม id
JOIN brands b ON b.id = p.brand_id	นำข้อมูล brand ที่สัมพันธ์กับ product มารวม

SELECT ได้ **rows** แต่ **INSERT/UPDATE/DELETE** ได้ **result header** เช่น **insertId** หรือ **affectedRows** ชื่อเหล่านี้ไม่ใช่ข้อมูลสินค้าเต็มแถว หลัง **INSERT** จึงมัก **SELECT** กลับเพื่อคืนข้อมูลจริง

id จาก URL มักเป็น string ส่วน id จากฐานมักเป็น number จึงอย่าใช้ row.id === req.params.id ตัดสินว่าพบข้อมูลหรือไม่ ให้ repository ค้นด้วย WHERE แล้วตรวจว่ามีแถวหรือไม่

อ้างอิง รูปแบบโค้ดจาก [S1-S7] **พร้อมคำอธิบายผู้เริ่มต้น**

บท 07 ตั้งค่าประจำตัวและอ่านฐานข้อมูลให้ตรง

สามค่าของผู้สอบ

practice-project/src/configs/student-config.js

```
export const student = {
  studentId: 'YOUR_STUDENT_ID',
  name: 'YOUR_NAME',
  secretKey: 'EXACT_SECRET'
}
```

เปลี่ยนข้อความภายในอัญประกาศทั้งสามค่าตามใบงาน แก้รหัสที่นี่แล้ว app จะใช้สร้าง base path ให้อัตโนมัติ รหัสนักศึกษาเก็บเป็น string เพื่อรักษาตัวเลขทุกหลัก อย่าคำนวณ secretKey เองถ้าโจทย์ให้ค่าที่ต้องตรงมาแล้ว

เชื่อมฐาน

ตัวอย่างค่าหลักใน configs/db-config.js

```
import mysql from 'mysql2/promise'

const db = mysql.createPool({
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: 'YOUR_MYSQL_PASSWORD',
  database: 'mobile_distributor_practice',
  dateStrings: true
})

export default db
```

ในห้องสอบให้แทน host, port, user, password และ database ด้วยค่าที่ได้รับ ไม่ใช่ยึดชื่อฐานฝึก ชุดไฟล์มี database/practice-only.sql สำหรับสร้างฐานฝึกใหม่เท่านั้น ห้ามนำ schema ฝึกไปทับฐานของอาจารย์

สร้างฐานฝึกก่อนรันคำสั่งตรวจฐาน

1. เปิด MySQL Workbench แล้วเข้า connection ของ MySQL ในเครื่องให้สำเร็จ

2. เลือก File > Open SQL Script แล้วเลือก practice-project/database/practice-only.sql

3. เลือกคำสั่งทั้งหมดแล้ว Execute เพื่อสร้างฐานและ seed ตาราง ขั้นตอนนี้ใช้เฉพาะชุดฝึก

4. รัน SELECT COUNT(*) FROM mobile_distributor_practice.products; ฐานฝึกที่สร้างใหม่ควรมี 3 แถว

ถ้าใช้ MySQL command line ให้เปิด Terminal พิมพ์ mysql -u root -p แล้วกรอกรหัสผ่าน เมื่อเห็น mysql> จึงใช้ SOURCE ตามตัวอย่างด้านล่าง เปลี่ยน path เป็นตำแหน่งจริง โดยใช้โฟลเดอร์ที่ไม่มีช่องว่างจะสะดวกกว่า

พิมพ์ที่ mysql> เปลี่ยน path ตัวอย่างให้ตรงเครื่อง

```
SOURCE C:/your-folder/practice-project/database/practice-only.sql;
```

รหัสผ่านใน db-config.js คือรหัส MySQL ไม่ใช่รหัสเข้า Windows ถ้าเครื่องตั้งรหัส MySQL ว่างจริง ให้ใช้ password: ''
การรัน SQL ฝึกซ้ำไม่รีเซ็ตทุกค่าที่เคยแก้ไว้ เพราะใช้ IF NOT EXISTS และ INSERT IGNORE

รันใน MySQL client หรือ SQL editor ไม่ใช่ไฟล์ JavaScript

```
SHOW DATABASES;
USE mobile_distributor_practice;
SHOW TABLES;
DESCRIBE brands;
DESCRIBE products;
SHOW CREATE TABLE products;
SELECT * FROM brands LIMIT 5;
```

คำใน **schema**	ผลต่อคำตอบ
PRIMARY KEY หรือ PK	คอลัมน์ระบุแถว ใช้ใน WHERE ไม่จำเป็นต้องชื่อ id เสมอ
AUTO_INCREMENT	ฐานสร้าง id ให้ INSERT แล้วใช้ result.insertId
UNIQUE	ค่าห้ามซ้ำ มักเกิด ER_DUP_ENTRY เมื่อชน
FOREIGN KEY หรือ FK	ค่าต้องอ้าง parent ที่มีจริง เช่น products.brand_id
NOT NULL	ฐานไม่อนุญาต null ต้องดูด้วยว่า field required/default อย่างไร
DEFAULT	ค่าเริ่มต้นของฐาน หากส่ง null เองไม่ได้แปลว่าใช้ default
DECIMAL	ราคาอาจถูก driver คืนเป็น string ต้อง map ตามชนิดที่ API กำหนด
DATE	วันที่ไม่มีเวลา ต้องคืน YYYY-MM-DD ถ้าโจทย์กำหนด

ชื่อที่ต้องจำในแบบฝึกนี้

คอลัมน์ **DB** และ **body** ในแบบฝึก	ชื่อใน **response**
brand_id	brandId
ram_gb	ramGb
storage_gb	storageGb
stock_quantity	stockQuantity
release_date	releaseDate
country_of_origin	countryOfOrigin
is_active	isActive

คอลัมน์ **DB** และ **body** ในแบบฝึก	ชื่อใน **response**
b.name AS brand_name ใน JOIN	brandName

SQL ใช้ **?** แทนค่า เช่น **id** หรือ **keyword** ได้ แต่ไม่ใช้ **?** แทนชื่อตารางหรือชื่อคอลัมน์เพื่อ **ORDER BY** ชื่อเหล่านั้นต้องมาจากโค้ดที่กำหนดไว้หรือ **whitelist**

อ้างอิง schema **สมมติใน** HTML [S1] **และจุดเปลี่ยนของ** template [S2-S7]

บท 08 ตั้ง response และ error ให้ตรงก่อนทำแต่ละข้อ

กรณี	**HTTP**	**response** ของแบบฝึก
GET สำเร็จ หรือ PUT สำเร็จ	200	{ success: true, data, timestamp }
POST สร้างสำเร็จ	201	{ success: true, data, timestamp }
DELETE สำเร็จ	204	ไม่มี body
ข้อมูลที่ส่งมาผิด	400	{ success: false, error: {code,message}, timestamp }
ไม่พบรายการ	404	error code ตาม resource เช่น PRODUCT_NOT_FOUND
ซ้ำ หรือ resource ยังถูกอ้าง	409	เช่น DUPLICATE_SKU หรือ RESOURCE_IN_USE
ความผิดพลาดที่ไม่ได้คาดไว้	500	ตรวจ log ฝั่ง server ไม่ใช่เปลี่ยนทุก error เป็น 404

mappers/api-response.js ในชุดฝึก

```
export function ok(res, data, status = 200) {
  return res.status(status).json({
    success: true,
    data,
    timestamp: new Date().toISOString()
  })
}
```

สังเกตตำแหน่ง argument อย่านำ helper มาปนกัน

```
// ชุดฝึกนี้ใช้ helper ok
ok(res, product, 201)

// template เดิมบางชุดใช้ apiResponse(res, data, pagination, status)
apiResponse(res, product, null, 201)

// DELETE 204
res.status(204).end()
```

โครง route ที่จัด error ชัดเจน

โค้ดเฉพาะ handler ต้องอยู่ในไฟล์ที่ import router, service, ok แล้ว

```
router.get('/:id', async (req, res, next) => {
  try {
    const product = await service.getById(req.params.id)
    return ok(res, product)
  } catch (error) {
    next(error)
  }
})
```

async ทำให้ใช้ await ได้ ส่วน catch ส่ง error ไปยัง handler กลาง error middleware ต้องมี 4 parameters คือ (err, req, res, next) และลงทะเบียนหลัง routes โค้ดเต็มในภาคผนวก A

ลำดับประกอบแอปแบบย่อ

```
app.use(express.json())
app.use(queryParser())
app.use(base + '/brands', brandRouter)
app.use(base + '/products', productRouter)
app.use(notFoundHandler)
app.use(errorHandler)
app.listen(3000)
```

newest helper ต้นฉบับบางตัวไม่มี **timestamp** ขณะที่ **HTML mock** และ **.md** ใช้ **timestamp** ให้เทียบ **JSON** ของข้อสอบจริงก่อนเพิ่มหรือลบ **field** คำว่า **success** และสถานะ **HTTP** เป็นคนละอย่าง ต้องถูกทั้งคู่

อ้างอิง รูปแบบ response **ของ** HTML [S1] **เทียบ** helpers [S2-S4] **และ** [S6-S7]

บท 09 Q1 อ่าน brand หนึ่งรายการและตอบเมื่อไม่พบ

โจทย์ฝึก GET /api/{ID}/brands/:id ต้องคืน 5 fields ได้แก่ id, name, countryOfOrigin, website, isActive และไม่ส่ง created_at ถ้าไม่พบคืน 404 พร้อม code BRAND_NOT_FOUND

1. repository ใช้ SELECT ... WHERE id = ? แล้วคืนแถวเดียวหรือ null

2. service ตรวจ null ถ้าไม่พบให้ throw 404 ก่อน map

3. mapper สร้าง object ใหม่เลือกเฉพาะ 5 fields

4. route รับ req.params.id เรียก service และใช้ ok ส่ง 200

repositories/brand-repo.js เฉพาะฟังก์ชัน

```
export async function findOne(id) {
  const [rows] = await db.query(
    'SELECT * FROM brands WHERE id = ?', [id]
  )
  return rows[0] ?? null
}
```

services/brand-service.js เฉพาะฟังก์ชัน

```
import { readId } from '../validators/id-validator.js'

export async function getById(id) {
  id = readId(id)
  const brand = await repo.findOne(id)
  if (!brand) {
    throw new AppError(
      "Brand with id '" + id + "' was not found",
      404, 'BRAND_NOT_FOUND'
    )
  }
  return toBrandDTO(brand)
}
```

mappers/brand-mapper.js

```
export function toBrandDTO(brand) {
  return {
    id: brand.id,
    name: brand.name,
    countryOfOrigin: brand.country_of_origin,
    website: brand.website,
    isActive: Number(brand.is_active) === 1
  }
}
```

ในชุดฝึก isActive เป็น boolean true/false ถ้าโจทย์จริงให้เป็น 0/1 ให้คืนตัวเลขตามโจทย์ ห้ามใช้ Boolean("0") เพื่อแปลง เพราะข้อความที่ไม่ว่างให้ true

ทดสอบ	ผลที่ต้องได้
GET brand id ที่มีจริง เช่น 1 ในฐานฝึก	200, data เป็น object, มี 5 fields
GET brand id ที่ไม่มี	404, error.code = BRAND_NOT_FOUND
ตรวจชื่อ field	countryOfOrigin และ isActive เป็น camelCase ไม่มี created_at

path ใน **app** เป็น **/api/{ID}/brands** แล้ว ใน **router** เขียนเพียง **/:id** หากใส่ **/brands/:id** ซ้ำจะกลายเป็น **/api/{ID}/brands/brands/:id**

readId เป็นกฎเสริมของชุดฝึก ตรวจ id ให้เป็นจำนวนเต็มบวกก่อนส่งไปเปรียบเทียบกับคอลัมน์ INT เพื่อกัน MySQL ตีความข้อความเช่น 1abc เป็น 1 ค่า id ผิดรูปตอบ 400 VALIDATION_ERROR ส่วน id รูปแบบถูกแต่ไม่มีแถวตอบ 404 ให้ปรับนโยบายนี้ถ้าโจทย์จริงกำหนดต่างกัน

อ้างอิง โจทย์จำลอง Q1 **ใน** [S1] **โค้ดฉบับฝึกในภาคผนวก** A

บท 10 Q2 รายการสินค้า การแบ่งหน้า และการกรอง

GET /api/{ID}/products?page=1&limit=10&search=phone&brandId=1 คืนรายการ Summary DTO 7 fields พร้อม pagination แบบฝึกนี้ page เริ่ม 1, limit เริ่ม 10, จำกัด limit สูงสุด 100 และ total ต้องนับหลังกรอง

คำนวณเลขหน้าให้ถูกก่อนเขียน SQL

สูตรจำ

```
const limit = Math.min(requestedLimit, 100)
const offset = (page - 1) * limit
const totalPages = Math.ceil(total / limit)
const hasNextPage = page < totalPages
const hasPrevPage = page > 1
```

page	**limit** ที่ใช้จริง	**offset**	ความหมาย
1	10	0	หยิบ 10 แถวแรก
2	10	10	ข้าม 10 แถวแล้วหยิบ 10
3	10	20	ข้าม 20 แถวแล้วหยิบ 10
2	100 เมื่อขอ 999	100	ต้องใช้ limit หลังจำกัดแล้ว

ถ้า total=23 และ limit=10 จะมี 3 หน้า หน้า 3 มี 3 แถว ถ้า total=0 จะได้ data: [] และ totalPages: 0 ห้ามทำ total หายเพียงเพราะ 0 เป็น falsy ถ้าขอ page เกินหน้าสุดท้าย แบบฝึกนี้คืน 200 และ array ว่างโดยยังบอก page ที่ขอ

เลือกนโยบายค่าผิดตามโจทย์

ชุดฝึกใช้ค่า default ถ้า page/limit ไม่ใช่จำนวนเต็มบวก และ cap limit ที่ 100 โค้ดอยู่ lib/pagination.js แต่ newest helper เดิมเปิด pagination เมื่อส่งทั้ง page และ limit และมี defaultLimit คนละค่า อย่าคัดลอกพฤติกรรมข้ามสัญญา หากข้อสอบบอกให้ตอบ 400 ต้องเปลี่ยนไป throw 400

หลักการของ parsePagination ดูไฟล์เต็มสำหรับ guard เพิ่ม

```
const rawPage = Number(query.page)
const rawLimit = Number(query.limit)
const page = Number.isSafeInteger(rawPage) && rawPage > 0
  ? rawPage : 1
const requested = Number.isSafeInteger(rawLimit) && rawLimit > 0
  ? rawLimit : 10
const limit = Math.min(requested, 100)
const offset = (page - 1) * limit
```

SQL และ COUNT ต้องใช้เงื่อนไขเดียวกัน

แนวทางสร้าง WHERE ใน repositories/product-repo.js

```
const conditions = []
const params = []
if (search) {
  conditions.push('(p.name LIKE ? OR p.sku LIKE ?)')
  params.push('%' + search + '%', '%' + search + '%')
}
if (brandId !== undefined) {
  conditions.push('p.brand_id = ?')
  params.push(brandId)
}
const where = conditions.length
  ? ' WHERE ' + conditions.join(' AND ') : ''
```

ส่วนอ่านรายการและนับจำนวน

```
const listSql = 'SELECT p.* FROM products p' + where +
  ' ORDER BY p.id ASC LIMIT ? OFFSET ?'
const countSql = 'SELECT COUNT(*) AS total FROM products p' + where

const [rows] = await db.query(listSql, [...params, limit, offset])
const [counts] = await db.query(countSql, params)
return { rows, total: Number(counts[0].total) }
```

ต้องมีวงเล็บรอบ (name LIKE ? OR sku LIKE ?) แล้วจึง AND brand_id = ? เพื่อให้ filter brand ใช้กับทั้ง name และ sku ค่าค้นหาถูกส่งผ่าน params ไม่เอาข้อความผู้ใช้มาต่อ SQL ตรง ๆ โดยแบบฝึกนี้ใช้ LIKE ตามปกติ ซึ่ง % และ _ ในคำค้นยังทำหน้าที่ wildcard

Summary DTO ของข้อ Q2

เลือกเฉพาะ 7 fields แม้ SQL จะอ่าน p.*

```
export function toSummaryDTO(product) {
  return {
    id: product.id,
    sku: product.sku,
    name: product.name,
    brandId: product.brand_id,
    category: product.category,
    price: Number(product.price),
    status: product.status
  }
}
```

หน้าตา response ที่ต้องตรวจ

ตัวอย่างเมื่อไม่พบรายการ timestamp จริงสร้างตอนตอบ

```
{
  "success": true,
  "data": [],
  "pagination": {
    "total": 0,
    "page": 1,
    "limit": 10,
    "totalPages": 0,
    "hasNextPage": false,
    "hasPrevPage": false
  },
  "timestamp": "2026-10-22T00:00:00.000Z"
}
```

ต้องลอง	กำลังตรวจอะไร
ไม่ส่ง query	default page=1, limit=10 ของแบบฝึก
page=2&limit=1	offset ทำให้ได้คนละแถวกับหน้า 1
search=คำที่มีใน name หรือ sku	filter ใช้ได้ทั้งสอง field
brandId=1	ผลทุกแถวต้องอ้างแบรนด์นั้น
search=คำที่ไม่มี	data ว่าง, total=0, pagination ยังอยู่
page=2&limit=999	limit=100 และ offset=100 ไม่ใช่ 999

อย่านับ **total** ด้วย **rows.length** เพราะนั่นคือจำนวนแถวของหน้าปัจจุบัน และอย่าเขียน **rows.totalItems** แล้วหวังว่า **JSON** จะมี **property** นี้เอง ชุดฝึกคืน **{ rows, total }** แยกกันเพื่อลดความสับสน

อ้างอิง โจทย์จำลอง Q2 [S1] **และสไลด์** Pagination/DTO **ใน** [S5]

บท 11 Q3 รายละเอียดสินค้าและข้อมูลจากอีกตาราง

GET /api/{ID}/products/:id คืน Detail DTO 13 fields โดยเพิ่ม brandName, ramGb, storageGb, color, stockQuantity และ releaseDate จาก Summary ถ้าไม่พบตอบ 404 PRODUCT_NOT_FOUND

repositories/product-repo.js หลักของ findOne

```
const sql =
  "SELECT p.*, b.name AS brand_name, " +
  "DATE_FORMAT(p.release_date, '%Y-%m-%d') AS release_date_text " +
  "FROM products p JOIN brands b ON b.id = p.brand_id " +
  "WHERE p.id = ?"
const [rows] = await db.query(sql, [id])
return rows[0] ?? null
```

p เป็นชื่อย่อของ products และ b เป็นชื่อย่อของ brands JOIN ใช้ p.brand_id หาชื่อแบรนด์จาก b.name แล้วตั้ง alias เป็น brand_name เพื่อให้ mapper อ่านได้ ชุดฝึกกำหนด FK แบบ NOT NULL และทุกสินค้าอ้าง brand จริง จึงใช้ JOIN ได้ หาก schema อนุญาตสินค้าไม่มี brand ให้พิจารณา LEFT JOIN ตามผลที่โจทย์ต้องการ

ข้อมูลที่คืน	แหล่งข้อมูลและการแปลง
id, sku, name, category, status	มาจาก products ชื่อเดิม
brandId	product.brand_id
brandName	product.brand_name จาก JOIN
price	Number(product.price)
ramGb, storageGb, color	ram_gb, storage_gb, color
stockQuantity	stock_quantity
releaseDate	release_date_text จาก DATE_FORMAT หรือ null

ตัวอย่าง data ของ detail เพื่อดูรูปแบบ ไม่ใช่ข้อมูลที่ต้องตรงกับ seed

```
{
  "id": 1,
  "sku": "PHONE-A",
  "name": "Practice Phone",
  "brandId": 1,
  "brandName": "Practice Brand",
  "category": "smartphone",
  "price": 12900,
  "ramGb": 8,
  "storageGb": 128,
  "color": "black",
  "stockQuantity": 10,
  "status": "AVAILABLE",
  "releaseDate": "2026-01-15"
}
```

อย่าส่งวันเป็น **timestamp** เต็มถ้าต้องการเพียง **YYYY-MM-DD** และอย่าเรียก **toISOString()** กับค่าที่เป็น **null** หรือ **string** โดยไม่ตรวจ ชุดฝึกใช้ **DATE_FORMAT** ใน **SQL** เพื่อได้ข้อความวันที่โดยไม่เลื่อนวันจาก **timezone**

หาก field ใน object เป็น undefined เมื่อส่ง JSON field นั้นอาจหายไป การขาด brandName จึงควรตรวจ JOIN และ alias ก่อนสงสัย route ส่วนไม่มีสินค้าให้ throw 404 ก่อนเข้า mapper เพื่อไม่ให้เกิด 500 จากการอ่าน property ของ null

อ้างอิง โจทย์จำลอง Q3 [S1] **และ** product DTO **ใน** [S5-S7]

บท 12 Q4 สร้างสินค้าและตรวจข้อมูลก่อนบันทึก

POST /api/{ID}/products รับ JSON body โดยบังคับ sku, name, brand_id, price ต้อง price > 0 และ stock_quantity >= 0 สร้างสำเร็จตอบ 201 พร้อม Detail DTO; SKU ซ้ำตอบ 409 DUPLICATE_SKU; brand ไม่มีตอบ 400 INVALID_BRAND

ตัวอย่าง request body ที่ส่งเข้ามาใช้ snake_case

```
{
  "sku": "MY-NEW-001",
  "name": "My Practice Phone",
  "brand_id": 1,
  "price": 15900,
  "stock_quantity": 0
}
```

ลำดับทำงาน

1. validator ตรวจ body เป็น object ตรวจ required และชนิดข้อมูล แล้วสร้าง object ใหม่เฉพาะ field ที่อนุญาต

2. service หา brand_id ในตาราง brands ถ้าไม่มีให้ตอบ INVALID_BRAND

3. repository INSERT ข้อมูลที่ผ่านการตรวจแล้ว

4. ใช้ result.insertId SELECT รายการที่สร้างกลับพร้อม JOIN

5. map เป็น Detail DTO แล้ว route ตอบ 201

6. ถ้า DB แจ้ง ER_DUP_ENTRY ให้แปลงเป็น DUPLICATE_SKU และสถานะ 409

รูปแบบ validation ที่ต้องเข้าใจ

ตัวอย่างเฉพาะแนวตรวจ โค้ดเต็มอยู่ validators/product-validator.js

```
if (body === null || typeof body !== 'object' || Array.isArray(body)) {
  throw new AppError('Body must be an object', 400, 'VALIDATION_ERROR')
}
if (typeof body.sku !== 'string' || body.sku.trim() === '') {
  throw new AppError('sku is required', 400, 'VALIDATION_ERROR')
}
if (typeof body.price !== 'number' ||
    !Number.isFinite(body.price) || body.price <= 0) {
  throw new AppError('Invalid price', 400, 'VALIDATION_ERROR')
}
```

ชุดฝึกเลือกกติกาเพิ่มเติมให้ตรวจ type ชัดเจน: ตัวเลขใน body ต้องเป็น JSON number ไม่รับ "15900", true หรือ [] ชื่อและ SKU ต้องเป็นข้อความไม่ว่างหลัง trim; field วันที่ต้องมีวันจริง; ram/storage เป็นจำนวนเต็มไม่ติดลบ กติกาเหล่านี้เป็นนโยบายของชุดฝึก หากโจทย์จริงให้แปลง numeric strings หรือมี enum ต่างออกไป ต้องเปลี่ยน validator ให้ตรง

stock_quantity = 0 ถูกต้อง จึงห้ามตรวจว่า **if (!body.stock_quantity)** ส่วน **Number(true)** ได้ **1** และ **Number("")** ได้ **0** การแปลงก่อนตรวจชนิดโดยไม่ตั้งใจทำให้ **input** ผิดผ่านได้

ชุดฝึกจำกัด price ไม่เกิน 99999999.99 และไม่เกิน 2 ตำแหน่งทศนิยม พร้อมจำกัด integer ไม่เกิน 2147483647 ให้สอดคล้องกับ DECIMAL(10,2) และ INT ของฐานฝึก เช่น 0.001 ถูกปฏิเสธเพื่อไม่ให้ฐานปัดเหลือ 0.00 ข้อจำกัดเหล่านี้ต้องปรับเมื่อ schema จริงต่างออกไป

บันทึกและอ่านกลับ

repositories/product-repo.js

```
export async function create(data) {
  const [result] = await db.query(
    'INSERT INTO products SET ?', [data]
  )
  return findOne(result.insertId)
}
```

ส่วนดัก duplicate ใน service ตัวอย่างข้อความปรับตาม spec

```
try {
  return toDetailDTO(await repo.create(data))
} catch (error) {
  if (error.code === 'ER_DUP_ENTRY') {
    throw new AppError('SKU already exists', 409, 'DUPLICATE_SKU')
  }
  throw error
}
```

SET ? แบบ object ใช้กับ mysql2.query ตามตัวอย่าง อย่าเปลี่ยนเป็น execute โดยส่ง object เดิม และอย่าส่ง req.body ตรงเข้า SQL ให้ whitelist เพื่อไม่ให้ id หรือ field ที่ฐานไม่มีหลุดเข้าไป

ทดสอบ	คาดหวังในแบบฝึก
body ครบ brand มีจริง sku ใหม่	201 และ data detail 13 fields
ส่ง SKU เดิมซ้ำ	409 DUPLICATE_SKU
ลบ name หรือใช้ชื่อช่องว่าง	400 VALIDATION_ERROR
price = 0 หรือค่าลบ	400 VALIDATION_ERROR
stock_quantity = 0	ผ่านเงื่อนไข stock
stock_quantity = -1	400 VALIDATION_ERROR
brand_id ที่ไม่มีจริง	400 INVALID_BRAND

อ้างอิง โจทย์จำลอง Q4 [S1] **และ** validator **ฉบับฝึกที่จัดทำเพิ่ม**

บท 13 Q5 แก้สินค้าและแยก full update จาก partial update

PUT /api/{ID}/products/:id ในแบบฝึกนี้เป็น full update ต้องส่ง required fields เหมือน POST ก่อนแก้ต้องตรวจว่ามี id นั้นจริง สำเร็จตอบ 200 พร้อม Detail DTO ของข้อมูลหลังแก้

ประเด็น	**full PUT** ในแบบฝึกนี้	**partial update** เมื่อโจทย์ระบุ
required fields	ต้องส่ง sku/name/brand_id/price	ตรวจเฉพาะ field ที่ส่ง ตามกติกาของโจทย์
field ที่ไม่ส่ง	optional กลับเป็น null หรือ default ของชุดฝึก	เก็บค่าเดิม
ส่ง {}	400 เพราะไม่มี required	มักปฏิเสธหรือไม่แก้อะไรตาม spec
ตัวอย่างในไฟล์แนบ	HTML mock	.md corrected ใช้การเก็บเฉพาะ fields ที่มี

ลำดับตรวจของชุดฝึกนี้

1. หา id เดิม ถ้าไม่มีให้ 404 PRODUCT_NOT_FOUND
2. ตรวจและ normalize body ด้วยฟังก์ชันเดียวกับ POST
3. ตรวจ brand_id ถ้าไม่มีให้ 400 INVALID_BRAND
4. UPDATE ตาม id จาก path แล้ว SELECT ค่าล่าสุดกลับ
5. SKU ชนสินค้าอีกชิ้นให้ 409 DUPLICATE_SKU; ใช้ SKU เดิมของแถวตัวเองได้

repository เฉพาะ UPDATE โดย service ตรวจว่ามี id แล้ว

```
export async function update(id, data) {
  await db.query(
    'UPDATE products SET ? WHERE id = ?', [data, id]
  )
  return findOne(id)
}
```

ตัวอย่าง request body ของ full PUT

```
{
  "sku": "MY-NEW-001",
  "name": "My Updated Phone",
  "brand_id": 1,
  "price": 14900,
  "stock_quantity": 5
}
```

ใช้ **id** จาก **req.params.id** ระบุแถว ไม่เอา **body.id** ไปเปลี่ยน **primary key** และอย่าใช้ **affectedRows** หรือ **changedRows** เพียงตัวเดียวตัดสินว่าไม่พบ เพราะส่งค่าที่เหมือนเดิมก็เป็นคำขอที่สำเร็จได้

ถ้าข้อสอบกำหนดลำดับตรวจต่างออกไป เช่น validate body ก่อนหา id ให้สลับลำดับตามนั้น Request ที่ผิดทั้ง id และ body อาจได้ error ต่างกันตามลำดับ และตัวตรวจอาจคาดผลที่เฉพาะเจาะจง

ทดสอบ	คาดหวัง
id มีจริงและ body ครบ	200, ได้ค่าที่แก้แล้ว
ส่งค่าเดิมซ้ำ	200 ไม่ควรกลายเป็น 404
id ไม่มี	404 PRODUCT_NOT_FOUND
id มีแต่ body มีแค่ price	400 ใน full PUT ของแบบฝึก
ใช้ SKU ของสินค้าอีกแถว	409 DUPLICATE_SKU
GET id เดิมหลังแก้	ฐานเก็บค่าที่เปลี่ยนจริง

อ้างอิง โจทย์จำลอง Q5 [S1] **เทียบคำตอบพื้นฐาน** [S7]

บท 14 Q6 ลบ brand และจัดการ foreign key

DELETE /api/{ID}/brands/:id ต้องตรวจ brand ก่อน ถ้าไม่มีตอบ 404 BRAND_NOT_FOUND ถ้า brand ยังมี product อ้างอยู่ตอบ 409 RESOURCE_IN_USE ถ้าลบได้ตอบ 204 ไม่มี response body

repositories/brand-repo.js

```
export async function remove(id) {
  const [result] = await db.query(
    'DELETE FROM brands WHERE id = ?', [id]
  )
  return result.affectedRows
}
```

service หลักการเดียวกับไฟล์เต็ม ข้อความต้องตรง spec หากกำหนด

```
import { readId } from '../validators/id-validator.js'

export async function remove(id) {
  id = readId(id)
  const brand = await repo.findOne(id)
  if (!brand) {
    throw new AppError('Brand not found', 404, 'BRAND_NOT_FOUND')
  }
  try {
    await repo.remove(id)
  } catch (error) {
    if (error.code === 'ER_ROW_IS_REFERENCED_2') {
      throw new AppError(
        'Brand is still used by products',
        409, 'RESOURCE_IN_USE'
      )
    }
    throw error
  }
}
```

route ส่งเฉพาะ status หลังลบ

```
router.delete('/:id', async (req, res, next) => {
  try {
    await service.remove(req.params.id)
    res.status(204).end()
  } catch (error) {
    next(error)
  }
})
```

สถานการณ์	ผลที่ต้องได้
brand ยังมีสินค้าอ้าง	409 RESOURCE_IN_USE และข้อมูลยังอยู่
brand ว่างไม่มีสินค้าอ้าง	204 ไม่มี body
ลบ brand เดิมอีกครั้ง	404 BRAND_NOT_FOUND
GET brand ที่เพิ่งลบ	404

ไม่ต้องปิด **foreign_key_checks** ไม่ต้องลบสินค้าลูกทิ้งเพื่อให้ลบ **brand** ผ่าน นั่นจะเปลี่ยนพฤติกรรมที่โจทย์ต้องการ และอย่าเรียก **ok(res, null, 204)** เพื่อหวังส่ง **JSON**

ถ้าฐานจริงกำหนด ON DELETE CASCADE แต่โจทย์ API ยังต้องห้ามลบ brand ที่มีสินค้า ต้องตรวจ COUNT ของลูกก่อนลบให้ได้ 409 ชุดฝึกใช้ FK แบบ RESTRICT จึงรับ error จากฐานได้โดยตรง ต้องอ่าน SHOW CREATE TABLE เพื่อรู้ว่าฐานจริงเลือกแบบไหน

อ้างอิง โจทย์จำลอง Q6 [S1] **และตัวอย่าง** CRUD **ใน** [S2-S7]

บท 15 เมื่อโจทย์จริงต่างจากแบบฝึก ต้องเปลี่ยนตรงไหน

สิ่งที่เปลี่ยน	จุดที่ต้องแก้
รหัส ชื่อ secretKey	config ของ student และ mount path ห้ามใช้ identity ของตัวอย่างเดิม
ชื่อ resource เช่น products เป็น offices	app mount + route/service/repo import และ SQL
ชื่อตารางหรือ PK	ทุก SELECT/UPDATE/DELETE ใน repo รวม alias และ key หลัง INSERT
request รับ brandId แทน brand_id	validator/service แปลง input -> DB object
response summary/full คนละ fields	mapper และ service ที่เลือก mapper
GET list ต้องมีทุกแถว ไม่แบ่งหน้า	ไม่ใส่ LIMIT/OFFSET และไม่ส่ง pagination หาก spec ไม่ต้องการ

สิ่งที่เปลี่ยน	จุดที่ต้องแก้
page/limit default หรือ cap ต่าง	pagination parser + test case
totalItems แทน total หรือ hasPreviousPage	response mapper ต้องใช้ชื่อ exact ตาม JSON
PUT แก้เฉพาะบางช่อง	ใช้ partial whitelist ไม่ normalize optional ทุกช่องเป็น null
DELETE ใช้ 200 พร้อม message	route ส่ง body ตามนั้น ไม่ใช้ 204
created response ต้องมี Location header	เพิ่ม res.location ตาม URL ที่โจทย์กำหนดก่อนส่ง 201

ถ้าโจทย์เป็น CRUD ของ brands แบบเอกสาร md

เอกสาร .md ที่แนบมี GET list, GET detail, POST, PUT และ DELETE สำหรับ brands/products ขณะที่ HTML แบบฝึกใช้ brands เพียง GET detail กับ DELETE หากโจทย์จริงขอ create/update brands ให้เพิ่ม route และ service/repo ของ brands ตามนั้น อย่าใช้ product validator ที่บังคับ sku กับตาราง brands

สูตรเสริมเฉพาะ repository ต้องมี validation/service/route และ mapper ตาม spec

```
// ตัวอย่าง repo ถ้า brands มี auto-increment id
export async function findAll() {
  const [rows] = await db.query('SELECT * FROM brands ORDER BY id')
  return rows
}

export async function create(data) {
  const [result] = await db.query('INSERT INTO brands SET ?', [data])
  return findOne(result.insertId)
}

export async function update(id, data) {
  await db.query('UPDATE brands SET ? WHERE id = ?', [data, id])
  return findOne(id)
}
```

ถ้าโจทย์ให้ GET /products?view=summary หรือ view=full ให้เลือก mapper ตาม view และตรวจว่า SQL ของรายการมี JOIN ที่จำเป็นสำหรับ full ด้วย การเปลี่ยน mapper อย่างเดียวไม่ได้ทำให้ brandName ปรากฏเอง

แนวเลือก DTO โดย rows ของ full ต้องมี field ที่ mapper ต้องใช้

```
const view = req.query.view ?? 'summary'
const data = rows.map(row =>
  view === 'full' ? toDetailDTO(row) : toSummaryDTO(row)
)
```

อย่าห่อ **data** ซ้ำ เช่น **service** คืน **{ data, total }** แล้ว **route** ส่ง **ok(res, result)** จนกลายเป็น **data.data** ถ้าใช้ **paginated** ให้ส่ง **result.data** และ **result.total** แยกตาม **helper**

อ้างอิง ความต่างของ [S1-S7] **และแนวปรับใช้**

บท 16 สูตรเสริมสำหรับโจทย์ที่ดัดแปลง

หัวข้อนี้เป็นสูตรสำหรับปรับตามโจทย์ ไม่ใช่ endpoint เพิ่มในโปรเจกต์ฝึก และไม่ได้หมายความว่าจะออกสอบ ให้ใช้เมื่อพบ requirement ตรงกันเท่านั้น โค้ดแต่ละกล่องเป็นส่วนที่ต้องวางใน layer ที่ระบุ

PK เป็นข้อความและไม่ได้สร้างอัตโนมัติ

repository ส่วน create เมื่อ PK มาจาก body

```
// ตัวอย่าง productlines มี PK ชื่อ productLine
const [result] = await db.query(
  'INSERT INTO productlines SET ?', [data]
)
const [rows] = await db.query(
  'SELECT * FROM productlines WHERE productLine = ?',
  [data.productLine]
)
return rows[0]
```

ใช้ key ที่ส่งมาอ่านกลับ ไม่ใช้ result.insertId กับ PK ข้อความ ต้องตรวจ required/duplicate ตามโจทย์ และถ้า path parameter มีช่องว่างให้ client URL-encode ค่า

Composite PK มีสองคอลัมน์

ตัวอย่างอ่าน payments ด้วย PK คู่

```
const [rows] = await db.query(
  'SELECT * FROM payments ' +
  'WHERE customerNumber = ? AND checkNumber = ?',
  [customerNumber, checkNumber]
)
```

อ่าน key ทั้งสองจาก path/body ตามโจทย์ ใช้ทั้งสองใน WHERE ของ GET/UPDATE/DELETE ถ้าสร้างใหม่ให้อ่านกลับด้วย key คู่นี้ ห้ามคาดว่า insertId ตัวเดียวระบุแถวได้

PATCH หรือ partial update ที่เปลี่ยนเฉพาะช่องที่ส่ง

ตัวอย่างใน service ที่รับ body object และตรวจ id แล้ว อนุญาตเพียง 2 fields

```
const data = {}
if (Object.hasOwn(body, 'price')) {
  if (typeof body.price !== 'number' ||
      !Number.isFinite(body.price) || body.price <= 0) {
    throw new AppError('Invalid price', 400, 'VALIDATION_ERROR')
  }
  data.price = body.price
}
if (Object.hasOwn(body, 'stock_quantity')) {
  const stock = body.stock_quantity
  if (typeof stock !== 'number' || !Number.isSafeInteger(stock) || stock < 0) {
    throw new AppError('Invalid stock', 400, 'VALIDATION_ERROR')
  }
  data.stock_quantity = stock
```

```
}
if (Object.keys(data).length === 0) {
  throw new AppError('No fields to update', 400, 'VALIDATION_ERROR')
}
await repo.update(id, data)
```

ใช้ Object.hasOwn เพื่อตรวจว่าผู้เรียกส่งช่องนั้นมาหรือไม่ ค่า 0 ยังถือว่าส่งมา อย่าเอา full validator ของ POST มาทำ PATCH เพราะมันจะบังคับ required ที่ไม่ได้แก้และอาจล้าง optional field เดิม

เรียงลำดับจาก query อย่างปลอดภัย

repository หรือ query builder ใช้ whitelist ของ column

```
const columns = { id: 'p.id', name: 'p.name', price: 'p.price' }
const sortBy = Object.hasOwn(columns, query.sortBy)
  ? columns[query.sortBy] : columns.id
const order = String(query.order).toLowerCase() === 'desc' ? 'DESC' : 'ASC'
const sql = 'SELECT p.* FROM products p ORDER BY ' + sortBy + ' ' + order
```

query.sortBy เอามาต่อ SQL ตรง ๆ ไม่ได้ และ ORDER BY ? ไม่ได้แทนชื่อคอลัมน์ตามที่ต้องการ เลือกชื่อ SQL จากรายการที่โค้ดอนุญาต หากโจทย์สั่งค่าผิดต้อง 400 ให้ปฏิเสธแทนการใช้ default

นับจำนวนสินค้าของแต่ละ brand

SQL สำหรับ aggregate

```
SELECT b.id, b.name, COUNT(p.id) AS productCount
FROM brands b
LEFT JOIN products p ON p.brand_id = b.id
GROUP BY b.id, b.name
ORDER BY b.id;
```

LEFT JOIN ทำให้ brand ที่ไม่มีสินค้าอยู่ในผลด้วย ใช้ COUNT(p.id) เพื่อให้ได้ 0 สำหรับ brand ว่าง ถ้าใช้ COUNT(*) อาจนับแถวด้านซ้ายเป็น 1 ชื่อ productCount และ DTO ต้องตามโจทย์

Nested route เช่นสินค้าของ brand

ตัวอย่าง route ต้อง import brandService, productService, paginated ให้ครบ

```
// mount router ไว้ที่ /api/{ID}/brands แล้ว
router.get('/:id/products', async (req, res, next) => {
  try {
    await brandService.getById(req.params.id)
    const query = { ...req.query, brandId: req.params.id }
    const result = await productService.list(req.pagination, query)
    paginated(res, result.data, result.total, req.pagination)
  } catch (error) { next(error) }
})
```

ตัวอย่างนี้ตรวจ parent ก่อน ถ้า brand ไม่มีจะ 404 ถ้ามีแต่ไม่มีสินค้าจะ 200 กับ array ว่าง การกำหนด brandId หลัง ...req.query ทำให้ผู้เรียกใช้ query เปลี่ยน brand จาก path ไม่ได้

วันที่ต้องเป็นวันจริง

regex ตรวจได้แค่หน้าตา เช่น 2026-02-31 ยังดูเหมือน YYYY-MM-DD ต้องแยกปีเดือนวัน สร้าง Date.UTC แล้วเทียบค่าปีเดือนวันกลับ ใช้ validDate ใน validator ของภาคผนวกได้ ชุดฝึกจำกัดปี 1000-9999 ตามฐานฝึก

ถ้ามีเส้นทางคงที่ เช่น **/summary** หรือ **/search** อยู่ใน **router** เดียวกับ **/:id** ให้ประกาศเส้นทางคงที่ก่อน **/:id** เพื่อไม่ให้คำว่า **summary** ถูกอ่านเป็น **id**

อ้างอิง รูปแบบฝึกเสริมใน HTML [S1] **และตัวอย่าง** schema/classicmodels **ใน** [S2-S5]

บท 17 ทดสอบแบบออฟไลน์และอ่านผลให้เป็น

ใช้ Postman

1. เริ่ม server ใน Terminal ก่อน แล้วเปิด Postman ที่เตรียมใช้ออฟไลน์ไว้

2. เลือก method เช่น GET, POST, PUT หรือ DELETE และใส่ URL เต็ม

3. สำหรับ POST/PUT ไป Body เลือก raw และ JSON ใส่ object โดยใช้ double quotes

4. ตรวจ Content-Type: application/json แล้วกด Send

5. ดูสถานะ HTTP ก่อน จากนั้นเทียบ JSON ทุก field และทำ GET ซ้ำเพื่อดูว่าฐานเปลี่ยนจริง

ใช้ Windows curl.exe หากไม่มี Postman

คำสั่งต่อไปนี้รันใน PowerShell หรือ CMD แก้ YOUR_STUDENT_ID ให้ตรงค่าจริง บน macOS/Linux ใช้ curl แทน curl.exe คำสั่งยาวหนึ่งคำสั่งต้องอยู่บรรทัดเดียวแม้ PDF จะแสดงสองบรรทัด แนะนำคัดลอกทั้งคำสั่งจาก HTML

Terminal อีกหน้าต่างหนึ่ง GET

```
curl.exe -i "http://localhost:3000/api/YOUR_STUDENT_ID/student-info"
curl.exe -i "http://localhost:3000/api/YOUR_STUDENT_ID/brands/1"
curl.exe -i "http://localhost:3000/api/YOUR_STUDENT_ID/products?page=1&limit=1"
```

ใช้ไฟล์ database/new-product.json ที่ให้มาในชุดฝึก โดยรันคำสั่งจากโฟลเดอร์ practice-project ที่มี package.json ไฟล์ JSON ใช้ double quotes และไม่มี comma เกินท้ายรายการ

POST จากโฟลเดอร์ practice-project

```
curl.exe -i -X POST "http://localhost:3000/api/YOUR_STUDENT_ID/products" -H "Content-Type:
application/json" --data-binary "@database/new-product.json"
```

ดู data.id ที่ได้จาก POST แล้วใช้ GET อ่าน id นั้นเพื่อตรวจว่าสร้างจริง ส่วนไฟล์ database/update-product.json เตรียมไว้สำหรับแก้ product 1 ของฐานฝึก โดยใช้ SKU PRACTICE-001 ของแถวเดิม จึงใช้ URL /products/1 ในคำสั่งต่อไปนี้

PUT product 1 ของฐานฝึก ใช้ไฟล์ update-product.json ที่ให้มา

```
curl.exe -i -X PUT "http://localhost:3000/api/YOUR_STUDENT_ID/products/1" -H "Content-Type:
application/json" --data-binary "@database/update-product.json"
```

DELETE brand 3 ที่ว่าง เมื่อเพิ่งสร้างฐานฝึกและยังไม่ได้แก้ seed

```
curl.exe -i -X DELETE "http://localhost:3000/api/YOUR_STUDENT_ID/brands/3"
```

-i ให้แสดง status/headers, -X เลือก method, -H ตั้ง header และ --data-binary @ไฟล์ ส่งเนื้อหาไฟล์ JSON ต้องครอบ URL ที่มี & ด้วยอัญประกาศเพราะ shell อาจแปล & เป็นคำสั่งพิเศษ

Matrix ที่ควรผ่านก่อนส่ง

กรณี	ผลที่ตรวจ
student-info	200 รหัส ชื่อ secret ตรง
Q1 brand มี / ไม่มี	200 DTO ถูก / 404 BRAND_NOT_FOUND
Q2 หน้าปกติ / กรอง / ไม่มีผล	200 array, metadata, total หลังกรองถูก
Q3 product มี / ไม่มี	200 detail 13 fields / 404 PRODUCT_NOT_FOUND
Q4 body ถูก / required หาย / sku ซ้ำ / brand ไม่มี	201 / 400 / 409 / 400
Q5 ค่าต่าง / ค่าเดิม / id ไม่มี / SKU ของแถวอื่น	200 / 200 / 404 / 409
Q6 brand ถูกอ้าง / brand ว่าง / ลบซ้ำ	409 / 204 ไม่มี body / 404
JSON เสีย เช่น comma เกิน	400 INVALID_JSON ในชุดฝึก
URL ที่ไม่ได้ประกาศ	404 ROUTE_NOT_FOUND ในชุดฝึก

คำสั่งทดสอบของแต่ละชุดไม่เหมือนกัน

ชุดฝึกใหม่: syntax check หนึ่งไฟล์ และ unit tests ที่ไม่ต้องต่อ DB

```
node --check src/app.js
npm test
```

node --check ตรวจเพียง syntax ของไฟล์ที่ระบุ ไม่ได้ทดสอบ import ทุกไฟล์หรือฐานข้อมูล npm test ของชุดฝึกทดสอบ validator/mapper/pagination เท่านั้น ไม่ได้ยืนยัน API เชื่อม MySQL สำเร็จ ต้องยิง request จริงอีกครั้งบนเครื่องที่เตรียมครบ

ใช้เฉพาะเมื่ออาจารย์แจกไฟล์นี้มาจริง และรันจาก project root

```
node tests/test-runner.mjs YOUR_STUDENT_ID
```

ถ้า Terminal อยู่ใน tests แล้ว คำสั่งจะเป็น node test-runner.mjs YOUR_STUDENT_ID ไม่ใช่ tests/tests/test-runner.mjs อย่าใช้คำสั่งนี้กับชุดแนบเดิมที่ไม่มีไฟล์ runner

อ้างอิง ชุดทดสอบแนะนำจาก [S1] **และคำสั่งใน** [S6-S7] **ที่ตรวจไฟล์จริงแล้ว**

บท 18 ตารางแก้ error เมื่อทำไม่ผ่าน

อ่านบรรทัด error แรกและชื่อไฟล์ก่อนแก้ ถ้า server ยังเริ่มไม่ได้ให้แก้ฝั่ง Terminal ก่อน ถ้าเริ่มได้แต่ request ผิดให้ดูทั้ง response และ log ของ server

อาการหรือข้อความ	สาเหตุที่ควรตรวจ	วิธีแก้ก่อน
ENOENT package.json	Terminal อยู่ผิดโฟลเดอร์	เปิดโฟลเดอร์ที่มี package.json
Missing script start หรือ dev	ต้นฉบับไม่มี script	ใช้ node src/app.js หรือ exam-app.js ที่แก้จริง
npm.ps1 running scripts is disabled	PowerShell บล็อก script wrapper	ใช้ npm.cmd ci/start/test ตามคำสั่งที่ต้องการ โดย ci ใช้ก่อนออฟไลน์
Cannot find package express/mysql2	dependencies ไม่มี	ใช้ชุดที่ติดตั้งไว้ก่อนสอบ; ชีทแทน package ไม่ได้
ERR_MODULE_NOT_FOUND เป็น path .js	import ชื่อไฟล์/โฟลเดอร์ไม่ตรง	ตรวจ singular/plural, -$id, ตัวพิมพ์ และ .js
Cannot use import outside a module	ไม่มี type module ใน package scope	ตรวจ "type": "module" ใน package.json
Unexpected token export	วงเล็บ/สตริงก่อนหน้าปิดไม่ครบ หรือ module ผิด	ตรวจบรรทัดก่อน export และ node --check
does not provide an export named	named/default import ไม่ตรง	export default ใช้ import x; export function ใช้ {x} หรือ * as
EADDRINUSE 3000	server เก่ายังเปิด port นี้	Ctrl+C ที่ Terminal server เก่าแล้วรันใหม่
ECONNREFUSED ตอนเรียก API	Node ไม่รันหรือ URL/port ผิด	ดู startup error และตรวจ port ตามโจทย์
ECONNREFUSED จาก MySQL	DB ไม่รันหรือ host/port ผิด	เปิด service DB และตรวจ config
ER_ACCESS_DENIED_ERROR	user/password ไม่ตรง	ใช้ค่าของเครื่องหรือใบงาน
ER_BAD_DB_ERROR	ชื่อฐานผิดหรือยังไม่สร้าง	ตรวจ SHOW DATABASES และ SQL ที่ต้องนำเข้า
ER_NO_SUCH_TABLE	ตารางไม่มีหรือเลือกฐานผิด	SHOW TABLES แล้วแก้ table/database
ER_BAD_FIELD_ERROR	ชื่อ column ผิดหรือใช้ camelCase ใน SQL	DESCRIBE ตาราง แล้ว map request เป็นชื่อ DB
ER_PARSE_ERROR	SQL ผิดหรือ data UPDATE ว่าง	ตรวจ SQL, comma, WHERE และ whitelist
req.body undefined	json middleware/Content-Type ไม่ครบ	express.json ก่อน routes และ Body เป็น raw JSON

อาการหรือข้อความ	สาเหตุที่ควรตรวจ	วิธีแก้ก่อน
Cannot GET URL	method/path/mount ไม่ตรง	รวม path จาก app กับ router ตรวจ ID และ prefix ซ้ำ
request ค้างไม่ตอบ	ไม่ได้ส่ง response หรือ error handler ว่าง	ทุก branch ต้องตอบหรือ next(error)
200 แต่ data เป็น {}	ลืม await หรือยังเป็น placeholder	await service และเปลี่ยน route เปล่า
500 เมื่อต้องเป็น 404	mapper อ่าน null ก่อนตรวจ	ตรวจว่า row มีจริงก่อน map
POST ได้ 200 แทน 201	ส่ง status ผิด argument ใน helper	apiResponse(res,data,null,201) หรือ ok(res,data,201)
total ไม่ตรงหลัง search	COUNT ไม่มี WHERE เหมือน SELECT	ใช้ buildFilter ชุดเดียวกัน
total=0 แล้ว metadata หาย	ใช้ if(total) หรือ property array หาย	แยก {rows,total} หรือเช็ก !== undefined
brandName หายจาก JSON	ไม่ได้ JOIN หรือ alias ไม่ตรง mapper	ตรวจ SELECT และชื่อ brand_name
price เป็นข้อความ	DECIMAL จาก driver	map Number(price) ถ้า spec ให้เป็น number
วันที่ผิดรูปแบบหรือเลื่อนวัน	Date/UTC แปลงไม่ตรง	DATE_FORMAT หรือ string date ตาม spec
แก้ไฟล์แล้วผลเดิม	ไม่ได้ save/restart หรือรันคนละ entry	Ctrl+S แล้ว restart ตัวที่ใช้จริง

ตรวจวงเล็บอย่างเร็ว

ทุก router.get/post/put/delete ต้องปิดด้วย })

```
router.get('/', async (req, res, next) => {
  try {
    // work here
  } catch (error) {
    next(error)
  }
})

export default router
```

อย่าแก้ด้วยการ catch แล้วคืน [] หรือส่ง 200 ทุกครั้ง เพราะจะซ่อนสาเหตุและทำให้กรณีที่ต้องตอบ 400/404/409 ไม่ตรง ตรวจ error เฉพาะชนิดที่รู้ แล้ว throw error เดิมสำหรับกรณีอื่น

อ้างอิง ข้อผิดพลาดที่ตรวจพบใน [S2-S7] **และแนววินิจฉัยตามไฟล์จริง**

บท 19 ตรวจงานก่อนส่งและหน้าจำเร็ว

ถ้าเวลาน้อย ให้ตรวจจุดนี้ก่อน

1. server เริ่มได้โดยไม่มี syntax/import error และอ่าน MySQL ได้จริง
2. รหัส ชื่อ secretKey, base path และ port ตรงโจทย์
3. ทุก endpoint ใช้ method ที่ถูกและ route ไม่ซ้ำ placeholder
4. GET detail คืน object; GET list คืน array; DTO fields ครบและไม่มีส่วนเกิน
5. POST ตอบ 201; PUT ตอบ 200; DELETE ตอบ 204 โดยไม่มี body ตามแบบฝึก
6. กรณีไม่พบตอบ 404 ไม่ใช่ null หรือ 500
7. validation และ duplicate/FK ส่ง code/message ตาม spec
8. pagination ใช้ offset จาก limit สุดท้าย และ COUNT หลังกรอง
9. แก้แล้ว save และรันไฟล์ที่ถูกจริง ปิด server เก่าที่ชน port
10. ตรวจชื่อไฟล์/โฟลเดอร์ที่จะส่งตามกติกา อย่าส่งค่าตัวอย่างคนอื่น

เห็นคำนี้	นึกถึงสิ่งนี้
GET /:id	req.params.id + WHERE PK = ? + rows[0] + 404
GET /	rows array + mapper รายตัว
page/limit	offset = (page - 1) * limit
total	COUNT หลัง WHERE เดียวกัน ไม่ใช่ rows.length
POST	req.body + validate + INSERT + insertId ถ้า auto + 201
PUT	ตรวจ id + full/partial ตามโจทย์ + UPDATE + SELECT กลับ
DELETE	ตรวจ id + FK + 204.end()
DTO	เลือก field เปลี่ยนชื่อ แปลงชนิด
brandName	JOIN brands + SQL alias
sku ซ้ำ	ER_DUP_ENTRY + 409
brand ไม่มี	ตรวจ parent + 400 ตาม mock
parent ถูกอ้าง	ER_ROW_IS_REFERENCED_2 + 409
PK ข้อความหรือคู่	ใช้ key จากโจทย์อ่านกลับ ห้ามพึ่ง insertId

เห็นคำนี้	นึกถึงสิ่งนี้
error middleware	(err, req, res, next) หลัง routes

ฝึกอย่างน้อยหนึ่งรอบก่อนสอบ

เปิดฐานฝึกและทำครบลำดับนี้ด้วยตัวเอง: student-info, GET brand, GET product, list หน้า 2, search ไม่พบ, POST สำเร็จ, POST ซ้ำ, PUT สำเร็จ, PUT id ไม่มี และ DELETE brand ที่ว่าง หลังจากนั้นเปลี่ยนชื่อ resource สมมติหนึ่งครั้งแล้วชี้ให้ได้ว่าต้องแก้ไฟล์ใดบ้าง การเปิดหาในชีทได้เร็วสำคัญกว่าการท่องโค้ดทั้งหน้า

อ้างอิง สรุปเกณฑ์จากแบบฝึก [S1] **และข้อผิดพลาดใน** template [S2-S7]

บท 20 แหล่งอ้างอิงและขอบเขตของตัวอย่าง

ใช้เนื้อหาในไฟล์แนบเป็นฐานอธิบายและตรวจเทียบกับ source code จริง ชื่อไฟล์ด้านล่างเป็นตัวระบุแหล่งอ้างอิงในท้ายบท ไม่จำเป็นต้องเปิดเว็บไซต์เพื่ออ่านชีท

รหัส	ไฟล์และส่วนที่ใช้
S1	INT161_Backend_Mock_Exam_With_Solutions (2).html: โจทย์จำลอง Q1-Q6, schema สมมติ, เฉลย และสูตรดัดแปลง
S2	newest-truely-midterm-express-esm-template.zip: package/lock, exam-* helpers, คู่มือ rename/import และหลาย resources
S3	truely-midterm-express-esm-template.zip: เปรียบเทียบกับ newest และโค้ด starter
S4	incase-of-that-shit-template-not-work.zip: fallback exam-crud-* และ pagination/response อีกแบบ
S5	Backend_ExamData-main.zip: mockExam, template ที่ซ้ำ และสไลด์ HTTP, REST, DB, Express, Error, Pagination/DTO
S6	express_practical_code_summary.md: อธิบาย layer และบรรทัดโค้ด
S7	express_practical_corrected_code_by_question.md: คำตอบพื้นฐาน student-info และ CRUD brands/products ที่มีสมมติฐาน schema

ตำแหน่งสไลด์ใน S5 ที่เกี่ยวข้อง: HTTP หน้า 10-15, REST หน้า 10-15, db-conection หน้า 7-10 และ 15-16, Express หน้า 7-17, Error Handling หน้า 12-15, Pagination and DTO หน้า 4-11 และ 14-23 เป็นเลขหน้าของ PDF

สิ่งที่กำหนดเพิ่มเพื่อให้ฝึกได้ครบ

• ฐาน mobile_distributor_practice และข้อมูลตัวอย่างสร้างใหม่ให้สอดคล้องกับ schema สมมติใน HTML ใช้เฉพาะฐานฝึก

• validator เข้มงวดกับ JSON number, ข้อความว่าง, วันที่จริง และ optional integer; status เป็นข้อความ default AVAILABLE ในฐานฝึก ไม่ใช่ enum ที่ยืนยันจากฐานสอบจริง รวมถึงช่วงตัวเลข จำนวนทศนิยม และ readId สำหรับ path parameter

• เพิ่ม request JSON และ unit tests สำหรับ helper ที่ไม่ต้องเชื่อม DB รวมถึง README ที่บอกวิธีเตรียมก่อน offline

• ภาคผนวกให้ source ครบชุด การนำไปใช้สอบต้องปรับให้ตรง spec, schema และข้อกำหนดโครงโปรเจกต์จริง

การตรวจโค้ดที่ทำได้ในสภาพแวดล้อมนี้เป็นการตรวจ **syntax/import paths** และทดสอบตรรกะที่ไม่ต่อฐาน ไม่ได้ติดตั้ง **dependencies** หรือรัน **API** กับ **MySQL** จริง จึงต้องทดลองชุดฝึกบนเครื่องที่พร้อมก่อนสอบ

ภาคผนวก A ต่อจากนี้แสดงไฟล์หลักทั้งหมดของชุดฝึก แบ่งด้วยชื่อ path เพื่อให้เปิดเทียบได้ ชุด ZIP มี package-lock.json, SQL, README, request JSON และ tests ประกอบด้วย

อ้างอิง ไฟล์ที่แนบมากับคำขอนี้และชุดฝึกที่จัดทำเพิ่ม

ภาคผนวก A โค้ดเต็มของชุดฝึก

ใช้ไฟล์ทั้งชุดจากโฟลเดอร์ practice-project เพื่อไม่ให้ import ขาด ภาคผนวกนี้แสดงเนื้อหาไฟล์หลักครบโดยไม่รวม lockfile และ tests ที่มีใน ZIP หากต้องคัดลอกโค้ด แนะนำ HTML หรือไฟล์ .js เพื่อรักษาบรรทัดและอัญประกาศ

A01 package.json

ไฟล์ package.json

```
{
  "name": "int161-offline-practice",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "description": "Standalone practice for the six-question mock; not the actual exam
template.",
  "scripts": {
    "start": "node src/app.js",
    "dev": "node --watch src/app.js",
    "test": "node --test tests/unit.test.js"
  },
  "engines": { "node": ">=18" },
  "dependencies": {
    "express": "5.2.1",
    "mysql2": "3.24.4"
  }
}
```

A02 src/configs/student-config.js

ไฟล์ src/configs/student-config.js

```
// แก้ 3 ค่านี้ให้เป็นของโจทย์ที่กำลังทำ; อย่าคัดลอกรหัสของคนอื่น
export const student = {
  studentId: 'YOUR_STUDENT_ID',
  name: 'YOUR_NAME',
  secretKey: 'EXACT_SECRET'
}
```

A03 src/configs/db-config.js

ไฟล์ src/configs/db-config.js

```
import mysql from 'mysql2/promise'

const db = mysql.createPool({
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: 'YOUR_MYSQL_PASSWORD',
  // ฐานจำลองใหม่เท่านั้น ไม่ใช่ฐานของข้อสอบจริง
  database: 'mobile_distributor_practice',
  waitForConnections: true,
  connectionLimit: 10,
  decimalNumbers: true,
  dateStrings: true
})

export default db
```

A04 src/app.js

ไฟล์ src/app.js

```
import express from 'express'
import { student } from './configs/student-config.js'
import brandRouter from './routes/brand-route.js'
import productRouter from './routes/product-route.js'
import { queryParser } from './middlewares/pagination-middleware.js'
import { notFoundHandler, errorHandler } from './middlewares/error-handler.js'
import { ok } from './mappers/api-response.js'

const app = express()
const base = '/api/' + student.studentId

app.use(express.json())
app.use(queryParser())

app.get(base + '/student-info', (req, res) => {
  ok(res, student)
})

app.use(base + '/brands', brandRouter)
app.use(base + '/products', productRouter)

app.use(notFoundHandler)
app.use(errorHandler)

app.listen(3000, () => {
  console.log('Practice server: http://localhost:3000' + base + '/student-info')
  console.log('Database: mobile_distributor_practice (separate practice database)')
})
```

A05 src/lib/pagination.js

ไฟล์ src/lib/pagination.js

```
// กฎของ mock นี้: page=1, limit=10, limit สูงสุด 100
export function parsePagination(query = {}) {
  const rawPage = Number(query.page)
  const rawLimit = Number(query.limit)
  const page = Number.isSafeInteger(rawPage) && rawPage > 0 ? rawPage : 1
  const requested = Number.isSafeInteger(rawLimit) && rawLimit > 0 ? rawLimit : 10
  const limit = Math.min(requested, 100)
  const offset = (page - 1) * limit
  // กันค่า offset เกินช่วงตัวเลขที่ JavaScript แทนได้แน่นอน
  if (!Number.isSafeInteger(offset)) return { page: 1, limit, offset: 0 }
  return { page, limit, offset }
}

export function paginationInfo(total, { page, limit }) {
  const totalPages = Math.ceil(total / limit)
  return {
    total,
    page,
    limit,
    totalPages,
    hasNextPage: page < totalPages,
    hasPrevPage: page > 1
  }
}
```

A06 src/middlewares/pagination-middleware.js

ไฟล์ src/middlewares/pagination-middleware.js

```
import { parsePagination } from '../lib/pagination.js'

export function queryParser() {
  return (req, res, next) => {
    req.pagination = parsePagination(req.query)
    next()
  }
}
```

A07 src/middlewares/error-handler.js

ไฟล์ src/middlewares/error-handler.js

```
export class AppError extends Error {
  constructor(message, statusCode = 500, code = 'INTERNAL_ERROR') {
    super(message)
    this.statusCode = statusCode
    this.code = code
  }
}

export function notFoundHandler(req, res, next) {
  next(new AppError('Route not found', 404, 'ROUTE_NOT_FOUND'))
```

```
}

export function errorHandler(err, req, res, next) {
  if (res.headersSent) return next(err)

  let status = err.statusCode || 500
  let code = err.code || 'INTERNAL_ERROR'
  let message = err.message || 'Unexpected server error'

  if (err.code === 'ER_DUP_ENTRY') {
    status = 409
    code = 'DUPLICATE_KEY'
  } else if (err.code === 'ER_NO_REFERENCED_ROW_2') {
    status = 400
    code = 'INVALID_REFERENCE'
  } else if (err.code === 'ER_ROW_IS_REFERENCED_2') {
    status = 409
    code = 'RESOURCE_IN_USE'
  } else if (err.type === 'entity.parse.failed') {
    status = 400
    code = 'INVALID_JSON'
    message = 'Request body must be valid JSON'
  }

  if (status === 500) console.error(err)

  return res.status(status).json({
    success: false,
    error: { code, message },
    timestamp: new Date().toISOString()
  })
}
```

A08 src/mappers/api-response.js

ไฟล์ src/mappers/api-response.js

```
import { paginationInfo } from '../lib/pagination.js'

export function ok(res, data, status = 200) {
  return res.status(status).json({
    success: true,
    data,
    timestamp: new Date().toISOString()
  })
}

export function paginated(res, data, total, pagination) {
  return res.status(200).json({
    success: true,
    data,
    pagination: paginationInfo(total, pagination),
    timestamp: new Date().toISOString()
  })
}
```

A09 src/mappers/brand-mapper.js

ไฟล์ src/mappers/brand-mapper.js

```
// Q1: มีเพียง 5 field; ไม่ส่ง created_at
export function toBrandDTO(brand) {
  return {
    id: brand.id,
    name: brand.name,
    countryOfOrigin: brand.country_of_origin,
    website: brand.website,
    isActive: Number(brand.is_active) === 1
  }
}
```

A10 src/mappers/product-mapper.js

ไฟล์ src/mappers/product-mapper.js

```
// Q2: Summary มีเพียง 7 field
export function toSummaryDTO(product) {
  return {
    id: product.id,
    sku: product.sku,
    name: product.name,
    brandId: product.brand_id,
    category: product.category,
    price: Number(product.price),
    status: product.status
  }
}

// Q3-Q5: Detail มี 13 field; brand_name มาจาก JOIN
export function toDetailDTO(product) {
  return {
    id: product.id,
    sku: product.sku,
    name: product.name,
    brandId: product.brand_id,
    brandName: product.brand_name,
    category: product.category,
    price: Number(product.price),
    ramGb: product.ram_gb,
    storageGb: product.storage_gb,
    color: product.color,
    stockQuantity: product.stock_quantity,
    status: product.status,
    releaseDate: product.release_date_text
  }
}

export function toSummaryDTOs(products) {
  return products.map(toSummaryDTO)
}
```

A11 src/validators/id-validator.js

ไฟล์ src/validators/id-validator.js

```
import { AppError } from '../middlewares/error-handler.js'

// กฎเสริมของแบบฝึก: path id ผิดรูปตอบ 400 ไม่ปล่อยให้ MySQL แปลง 1abc เป็น 1
export function readId(raw) {
  const id = Number(raw)
  if (typeof raw !== 'string' || !/^\d+$/.test(raw) ||
      !Number.isSafeInteger(id) || id <= 0 || id > 2147483647) {
    throw new AppError('id must be a positive integer', 400, 'VALIDATION_ERROR')
  }
  return id
}
```

A12 src/validators/product-validator.js

ไฟล์ src/validators/product-validator.js

```
import { AppError } from '../middlewares/error-handler.js'

const MAX_INT = 2147483647
const MAX_PRICE = 99999999.99 // DECIMAL(10,2) ของฐานจำลอง

function invalid(message) {
  throw new AppError(message, 400, 'VALIDATION_ERROR')
}

function text(value, field, maximum, required = false) {
  if (value === undefined || value === null) {
    if (required) invalid("Field '" + field + "' is required")
    return null
  }
  if (typeof value !== 'string') invalid(field + ' must be a string')
  const result = value.trim()
  if (required && result.length === 0) invalid("Field '" + field + "' is required")
  if (result.length > maximum) invalid(field + ' is too long')
  return result
}

function optionalInteger(value, field) {
  if (value === undefined || value === null) return null
  if (typeof value !== 'number' || !Number.isSafeInteger(value) || value < 0 || value >
MAX_INT) {
    invalid(field + ' must be a non-negative integer or null')
  }
  return value
}

export function validDate(value) {
  if (typeof value !== 'string' || !/^\d{4}-\d{2}-\d{2}$/.test(value)) return false
  const [year, month, day] = value.split('-').map(Number)
  if (year < 1000 || year > 9999) return false
  const date = new Date(Date.UTC(year, month - 1, day))
  return date.getUTCFullYear() === year &&
    date.getUTCMonth() === month - 1 && date.getUTCDate() === day
```

```
}

// Q4/Q5: POST และ full PUT ใช้กฎเดียวกันในแบบฝึกนี้
// ตัวเลขใน JSON ต้องเป็น number: 1200 ใช้ได้, "1200"/true/[] ใช้ไม่ได้
export function normalizeAndValidate(body) {
  if (body === null || typeof body !== 'object' || Array.isArray(body)) {
    invalid('Request body must be a JSON object')
  }

  const sku = text(body.sku, 'sku', 50, true)
  const name = text(body.name, 'name', 150, true)

  if (body.brand_id === undefined || body.brand_id === null || body.brand_id === '') {
    invalid("Field 'brand_id' is required")
  }
  if (body.price === undefined || body.price === null || body.price === '') {
    invalid("Field 'price' is required")
  }

  const price = body.price
  if (typeof price !== 'number' || !Number.isFinite(price) || price <= 0) {
    invalid('Price must be greater than zero')
  }
  if (price > MAX_PRICE) invalid('Price exceeds the practice database limit')
  if (Number(price.toFixed(2)) !== price) invalid('Price must have at most two decimal places')

  const stock = body.stock_quantity === undefined ? 0 : body.stock_quantity
  if (typeof stock !== 'number' || !Number.isSafeInteger(stock) || stock < 0 || stock >
MAX_INT) {
    invalid('Stock quantity must be a non-negative integer')
  }

  const brandId = body.brand_id
  if (typeof brandId !== 'number' || !Number.isSafeInteger(brandId) || brandId <= 0 || brandId
> MAX_INT) {
    invalid('brand_id must be a positive integer')
  }

  const releaseDate = body.release_date ?? null
  if (releaseDate !== null && !validDate(releaseDate)) {
    invalid('release_date must be a valid YYYY-MM-DD date or null')
  }

  // Whitelist: ไม่มี id แม้ผู้ใช้ส่ง id มาด้วย
  // full PUT: optional field ที่ไม่ส่งจะกลับเป็น null/default ตามแบบฝึกนี้
  return {
    sku,
    name,
    brand_id: brandId,
    category: text(body.category, 'category', 30),
    price,
    ram_gb: optionalInteger(body.ram_gb, 'ram_gb'),
    storage_gb: optionalInteger(body.storage_gb, 'storage_gb'),
    color: text(body.color, 'color', 50),
    stock_quantity: stock,
    status: text(body.status ?? 'AVAILABLE', 'status', 20),
    release_date: releaseDate
  }
```

```
}
```

A13 src/repositories/brand-repo.js

ไฟล์ src/repositories/brand-repo.js

```
import db from '../configs/db-config.js'

export async function findOne(id) {
  const [rows] = await db.query('SELECT * FROM brands WHERE id = ?', [id])
  return rows[0] ?? null
}

export async function remove(id) {
  const [result] = await db.query('DELETE FROM brands WHERE id = ?', [id])
  return result.affectedRows
}
```

A14 src/repositories/product-repo.js

ไฟล์ src/repositories/product-repo.js

```
import db from '../configs/db-config.js'

// Q2: SQL list และ count ใช้เงื่อนไขเดียวกัน
function buildFilter({ search, brandId }) {
  const conditions = []
  const params = []
  if (search) {
    conditions.push('(p.name LIKE ? OR p.sku LIKE ?)')
    const keyword = '%' + search + '%'
    params.push(keyword, keyword)
  }
  if (brandId !== undefined) {
    conditions.push('p.brand_id = ?')
    params.push(brandId)
  }
  const where = conditions.length > 0 ? ' WHERE ' + conditions.join(' AND ') : ''
  return { where, params }
}

export async function findAll(pagination, filters) {
  const { where, params } = buildFilter(filters)
  const listSql = 'SELECT p.* FROM products p' + where +
    ' ORDER BY p.id ASC LIMIT ? OFFSET ?'
  const countSql = 'SELECT COUNT(*) AS total FROM products p' + where
  const [rows] = await db.query(listSql, [...params, pagination.limit, pagination.offset])
  const [counts] = await db.query(countSql, params)
  return { rows, total: Number(counts[0].total) }
}

const FIND_ONE =
  "SELECT p.*, b.name AS brand_name, " +
  "DATE_FORMAT(p.release_date, '%Y-%m-%d') AS release_date_text " +
  "FROM products p JOIN brands b ON b.id = p.brand_id WHERE p.id = ?"
```

```
export async function findOne(id) {
  const [rows] = await db.query(FIND_ONE, [id])
  return rows[0] ?? null
}

export async function create(data) {
  // SET ? รับ object ได้กับ mysql2.query; อย่าเปลี่ยนเป็น execute โดยตรง
  const [result] = await db.query('INSERT INTO products SET ?', [data])
  return findOne(result.insertId)
}

export async function update(id, data) {
  await db.query('UPDATE products SET ? WHERE id = ?', [data, id])
  return findOne(id)
}
```

A15 src/services/brand-service.js

ไฟล์ src/services/brand-service.js

```
import * as repo from '../repositories/brand-repo.js'
import { toBrandDTO } from '../mappers/brand-mapper.js'
import { AppError } from '../middlewares/error-handler.js'
import { readId } from '../validators/id-validator.js'

function notFound(id) {
  return new AppError("Brand with id '" + id + "' was not found", 404, 'BRAND_NOT_FOUND')
}

export async function getById(id) {
  id = readId(id)
  const brand = await repo.findOne(id)
  if (!brand) throw notFound(id)
  return toBrandDTO(brand)
}

export async function remove(id) {
  id = readId(id)
  if (!(await repo.findOne(id))) throw notFound(id)
  try {
    await repo.remove(id)
  } catch (error) {
    if (error.code === 'ER_ROW_IS_REFERENCED_2') {
      throw new AppError(
        'Cannot delete brand because products still reference it',
        409,
        'RESOURCE_IN_USE'
      )
    }
    throw error
  }
}
```

A16 src/services/product-service.js

ไฟล์ src/services/product-service.js

```
import * as repo from '../repositories/product-repo.js'
import * as brandRepo from '../repositories/brand-repo.js'
import { toSummaryDTOs, toDetailDTO } from '../mappers/product-mapper.js'
import { AppError } from '../middlewares/error-handler.js'
import { normalizeAndValidate } from '../validators/product-validator.js'
import { readId } from '../validators/id-validator.js'

function notFound(id) {
  return new AppError("Product with id '" + id + "' was not found", 404, 'PRODUCT_NOT_FOUND')
}

async function checkBrand(brandId) {
  if (!(await brandRepo.findOne(brandId))) {
    throw new AppError('Referenced brand does not exist', 400, 'INVALID_BRAND')
  }
}

// Q2: query string เป็นข้อความ; กัน search ซ้ำจนกลายเป็น array
export async function list(pagination, query) {
  if (query.search !== undefined && typeof query.search !== 'string') {
    throw new AppError('search must be a single string', 400, 'VALIDATION_ERROR')
  }
  if (query.brandId !== undefined && typeof query.brandId !== 'string') {
    throw new AppError('brandId must be a single value', 400, 'VALIDATION_ERROR')
  }
  const filters = { search: (query.search ?? '').trim() }
  if (query.brandId !== undefined && query.brandId !== '') {
    const brandId = Number(query.brandId)
    if (!Number.isSafeInteger(brandId) || brandId <= 0) {
      throw new AppError('brandId must be a positive integer', 400, 'VALIDATION_ERROR')
    }
    filters.brandId = brandId
  }
  const { rows, total } = await repo.findAll(pagination, filters)
  return { data: toSummaryDTOs(rows), total }
}

export async function getById(id) {
  id = readId(id)
  const product = await repo.findOne(id)
  if (!product) throw notFound(id)
  return toDetailDTO(product)
}

// Q4: validate -> ตรวจ brand -> insert -> คืน Detail DTO
export async function create(body) {
  const data = normalizeAndValidate(body)
  await checkBrand(data.brand_id)
  try {
    return toDetailDTO(await repo.create(data))
  } catch (error) {
    if (error.code === 'ER_DUP_ENTRY') {
      throw new AppError("SKU '" + data.sku + "' already exists", 409, 'DUPLICATE_SKU')
    }
```

```
    if (error.code === 'ER_NO_REFERENCED_ROW_2') {
      throw new AppError('Referenced brand does not exist', 400, 'INVALID_BRAND')
    }
    throw error
  }
}

// Q5: ตรวจ id ก่อน -> full validation -> brand -> update -> select กลับ
export async function update(id, body) {
  id = readId(id)
  if (!(await repo.findOne(id))) throw notFound(id)
  const data = normalizeAndValidate(body)
  await checkBrand(data.brand_id)
  try {
    return toDetailDTO(await repo.update(id, data))
  } catch (error) {
    if (error.code === 'ER_DUP_ENTRY') {
      throw new AppError(
        "SKU '" + data.sku + "' already belongs to another product",
        409,
        'DUPLICATE_SKU'
      )
    }
    if (error.code === 'ER_NO_REFERENCED_ROW_2') {
      throw new AppError('Referenced brand does not exist', 400, 'INVALID_BRAND')
    }
    throw error
  }
}
```

A17 src/routes/brand-route.js

ไฟล์ src/routes/brand-route.js

```
import express from 'express'
import * as service from '../services/brand-service.js'
import { ok } from '../mappers/api-response.js'

const router = express.Router()

// Q1: URL จริงมาจาก prefix ใน app.js + /:id
router.get('/:id', async (req, res, next) => {
  try {
    ok(res, await service.getById(req.params.id))
  } catch (error) {
    next(error)
  }
})

// Q6: ไม่เรียก ok เพราะ 204 ต้องไม่มี JSON body
router.delete('/:id', async (req, res, next) => {
  try {
    await service.remove(req.params.id)
    res.status(204).end()
  } catch (error) {
    next(error)
  }
```

```
})

export default router
```

A18 src/routes/product-route.js

ไฟล์ src/routes/product-route.js

```
import express from 'express'
import * as service from '../services/product-service.js'
import { ok, paginated } from '../mappers/api-response.js'

const router = express.Router()

// Q2
router.get('/', async (req, res, next) => {
  try {
    const result = await service.list(req.pagination, req.query)
    paginated(res, result.data, result.total, req.pagination)
  } catch (error) {
    next(error)
  }
})

// Q3
router.get('/:id', async (req, res, next) => {
  try {
    ok(res, await service.getById(req.params.id))
  } catch (error) {
    next(error)
  }
})

// Q4
router.post('/', async (req, res, next) => {
  try {
    ok(res, await service.create(req.body), 201)
  } catch (error) {
    next(error)
  }
})

// Q5
router.put('/:id', async (req, res, next) => {
  try {
    ok(res, await service.update(req.params.id, req.body))
  } catch (error) {
    next(error)
  }
})

export default router
```

A19 database/practice-only.sql

ไฟล์ database/practice-only.sql

```
-- PRACTICE ONLY: schema created for the six-question HTML mock.
-- This is NOT the SQL dump of the actual exam. No DROP statements.
-- Uses only mobile_distributor_practice, never mobile_distributor/classicmodels.

CREATE DATABASE IF NOT EXISTS mobile_distributor_practice
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE mobile_distributor_practice;

CREATE TABLE IF NOT EXISTS brands (
  id INT NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  country_of_origin VARCHAR(100) NULL,
  website VARCHAR(255) NULL,
  is_active TINYINT NOT NULL DEFAULT 1,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_brands_name (name)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS products (
  id INT NOT NULL AUTO_INCREMENT,
  sku VARCHAR(50) NOT NULL,
  name VARCHAR(150) NOT NULL,
  brand_id INT NOT NULL,
  category VARCHAR(30) NULL,
  price DECIMAL(10,2) NOT NULL,
  ram_gb INT NULL,
  storage_gb INT NULL,
  color VARCHAR(50) NULL,
  stock_quantity INT NOT NULL DEFAULT 0,
  status VARCHAR(20) NOT NULL DEFAULT 'AVAILABLE',
  release_date DATE NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_products_sku (sku),
  CONSTRAINT fk_products_brand FOREIGN KEY (brand_id)
    REFERENCES brands (id) ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB;

-- INSERT IGNORE keeps existing practice rows; it does not reset your earlier changes.
INSERT IGNORE INTO brands
  (id, name, country_of_origin, website, is_active)
VALUES
  (1, 'Practice One', 'Thailand', 'https://example.invalid/one', 1),
  (2, 'Practice Two', 'Japan', NULL, 1),
  (3, 'Empty Practice Brand', NULL, NULL, 0);

INSERT IGNORE INTO products
  (id, sku, name, brand_id, category, price, ram_gb, storage_gb,
   color, stock_quantity, status, release_date)
VALUES
  (1, 'PRACTICE-001', 'Practice Phone Alpha', 1, 'PHONE', 12500.00,
   8, 128, 'Black', 5, 'AVAILABLE', '2026-01-15'),
  (2, 'PRACTICE-002', 'Practice Phone Beta', 1, 'PHONE', 14900.00,
   12, 256, 'White', 3, 'AVAILABLE', '2026-02-10'),
```

```
  (3, 'PRACTICE-003', 'Practice Tablet', 2, 'TABLET', 9900.00,
   8, 128, 'Silver', 0, 'AVAILABLE', '2026-03-01');
```

A20 database/new-product.json

ไฟล์ database/new-product.json

```
{
  "sku": "PRACTICE-NEW-001",
  "name": "New Practice Phone",
  "brand_id": 1,
  "price": 15900,
  "category": "PHONE",
  "ram_gb": 8,
  "storage_gb": 256,
  "color": "Blue",
  "stock_quantity": 10,
  "status": "AVAILABLE",
  "release_date": "2026-04-20"
}
```

A21 database/update-product.json

ไฟล์ database/update-product.json

```
{
  "sku": "PRACTICE-001",
  "name": "Practice Phone Alpha Updated",
  "brand_id": 1,
  "price": 12000,
  "category": "PHONE",
  "ram_gb": 8,
  "storage_gb": 128,
  "color": "Black",
  "stock_quantity": 4,
  "status": "AVAILABLE",
  "release_date": "2026-01-15"
}
```